

EchoForge: A Strict-Open Synthetic Benchmark for Low-Altitude UAS Radar and Multimodal Sensing

Jeppson Taylor, Lead Author
NeverHuman Research Group

Abstract—EchoForge is a strict-open synthetic benchmark for low-altitude uncrewed aerial system (UAS) radar and multimodal sensing research. The paper-facing positive class is a fixed-wing pusher-prop public proxy with explicit uncertainty; it is not a measured-platform claim and is not a fielded-sensor surrogate. The final benchmark run contains 10,000 scenario groups, 50 positive groups, and 30,000 phase records. Candidate discovery, thresholding, and calibration use train/CV rows only; the blind group-locked holdout is scored once after selection lock. Primary KPI: LCB95 Recall@ $\leq 1\%$ FPR. The accepted human-engineered prior fusion baseline is the comparator. On the 4,500-record blind holdout, with 24 positive records from eight positive scenario groups, Engineered Intelligence (EI) reaches point Recall@ $\leq 1\%$ FPR of **0.833 vs 0.292 (+54.2 pp, +185.7%, 2.85x)** and LCB95 Recall@ $\leq 1\%$ FPR of **0.713 vs 0.083 (+63.0 pp, +760%, 7.60x)**. EI also raises average precision from **0.128 to 0.830 (+0.702, about +547%)** and cuts selected-threshold false positives from 49 to 3 (93.9% fewer false alarms). **EI ROC AUC is 0.916, below prior fusion's 0.938**, so the claim is improved low-FPR operating behavior for this synthetic public-proxy benchmark, not universal rank dominance. These results are reproducible benchmark evidence under declared assumptions, not measured truth, named-platform equivalence, or operational performance.

Index Terms—Synthetic radar, UAS detection, public proxy, micro-Doppler, sensor fusion, calibration, reproducibility, compare-only measured anchor.

I. CLAIM BOUNDARY AND CONTRIBUTIONS

EchoForge is intentionally narrower than a measured radar collection. It publishes public-proxy signatures and detector-facing products with explicit uncertainty and validation tiers, while keeping generated arrays, solver outputs, and measured raw files out of Git. The radar and detection framing follows standard signal-processing and CFAR practice rather than a sensor-surrogate claim [1]–[3]. The paper-facing positive class is named *fixed-wing pusher-prop public proxy*. No measured-platform truth, proprietary-equivalent behavior, fielded-system equivalence, or classified fidelity is claimed for any object, platform, or sensor.

The paper has one main experiment: compare accepted human-engineered radar/cue baselines and prior fusion against a locked EI sparse late-fusion artifact on the same blind, group-locked synthetic holdout. The contributions are:

- a strict-open radar benchmark lane with explicit waveform, range-Doppler, clutter/noise/RFI, acoustic, passive-RF, and detector-view assumptions;
- a rare-positive, low-false-alarm evaluation protocol centered on LCB95 Recall@ $\leq 1\%$ FPR, with AP, ROC AUC, calibration, and false-alarm families as guardrails;

- a detector/fusion benchmark that moves from human detectors to accepted prior fusion and then to EI under one split discipline;
- a generated evidence and source appendix that exposes model cards, CLI commands, source provenance, EI search traces, and no-holdout-selection guarantees.

A. Core Experiment

The core experiment compares classical radar and cue detector branches, tabular and sequence ML baselines, an accepted human-engineered prior fusion baseline, and EI sparse calibrated late fusion. Scenario groups are locked before phase expansion, train/CV rows are used for candidate discovery and calibration, and the blind holdout is scored once after selection lock. The claim tested is deliberately narrow: under the declared synthetic public-proxy generator and group-locked split, EI improves the low-FPR operating point over the accepted human-engineered prior fusion baseline. This comparator is not a straw baseline; it is the strongest non-EI late-fusion lane assembled from radar/cue branches under the same split and train/CV thresholding policy.

Benchmark positioning is compare-only. KTH provides a public 77 GHz FMCW measured anchor for drone/bird/human observable-shape checks; Open Radar and RAD-DAR/RDRD provide public radar and range-Doppler reference points; and public UAS micro-Doppler studies bound the observable families used for small aerial objects [4]–[10]. EchoForge does not replace those measured collections. It provides a strict-open synthetic lane for controlled hard negatives, leakage checks, and low-FPR detector comparison when measured positive-class truth is not available.

II. DATA PROCESSING AND EVIDENCE FLOW

EchoForge should be read as a pipeline, not as a ledger of disconnected artifacts. Figure 2 is the reviewer-facing trace: scenario groups are sampled, expanded into phase records, converted into synthetic radar/cue products, materialized as detector-view features through a denylisted schema, searched/calibrated on train/CV rows, locked, and scored once on blind holdout. The paper evidence manifest is labeled *tier1-final*, and the key generated rows are `data_processing_trace_rows.csv`, `simulation_best_practice_rows.csv`, `phase_method_ladder_rows.csv`, `ei_evolution_trace.csv`, and `source_appendix_hashes.csv`.

TABLE I
CORE EXPERIMENT ROADMAP

Lane	What is modeled	Why it is in the paper	Reader-facing conclusion
Sensor branches	X/Ku radar, S-band radar, GBAD cueing, acoustic cueing, and passive-RF provenance	Shows component evidence before fusion	Branches are sanity checks and failure-slice baselines
Prior ML	Tabular and sequence learners trained only on detector-facing features	Tests ordinary learned controls under the same split	Useful controls, not the accepted-practice comparator
Accepted fusion	Human-engineered layered fusion with train/CV thresholding	Represents accepted human-engineered prior fusion	Main comparator for EI
EI sparse fusion	Train/CV candidate discovery, sparse non-negative component fusion, monotone odds calibration, and one locked holdout pass	Tests whether EI improves the low-FPR operating point	Main challenger; claims are limited to this synthetic public-proxy benchmark

EchoForge radar evidence stack

Public-proxy priors, synthetic sensing, detector views, and validation evidence stay separated by claim boundary.

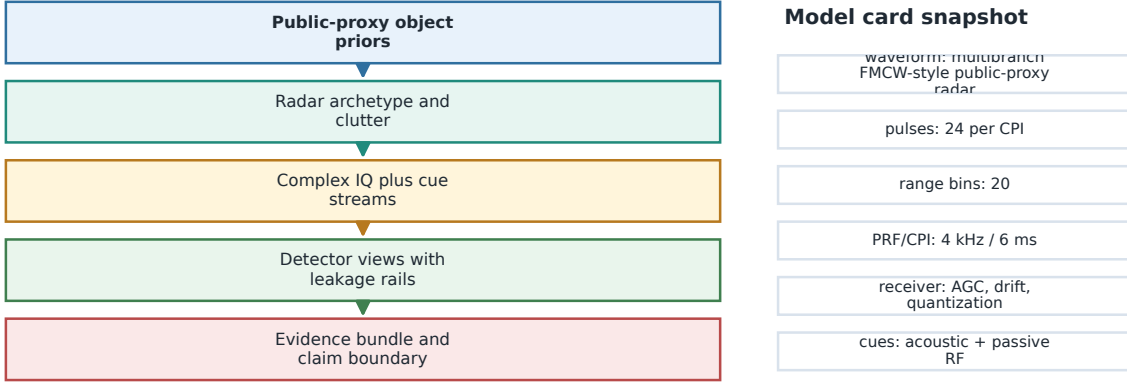


Fig. 1. Radar signal chain and claim boundary. Public-proxy object priors, radar archetype assumptions, clutter and artifact priors, detector views, and evidence outputs are separated so the benchmark can be audited without conflating synthetic products with measured truth.

III. SIMULATION AND MONTE CARLO

EchoForge uses a clean-room signal-chain proxy rather than a proprietary sensor surrogate. The received-power proxy is

$$P_r \propto \frac{P_t G_t G_r \lambda^2 \sigma(\theta, f, \phi)}{(4\pi)^3 R^4 L}, \quad (1)$$

where σ is an aspect- and frequency-dependent public-proxy RCS term, R is slant range, and L is aggregate loss. Complex IQ is generated as a phase-stable target term plus clutter, interference, and receiver impairment terms,

$$x_{m,n} = a_{m,n} e^{j(2\pi f_d m T_r + \varphi_n)} + c_{m,n} + \epsilon_{m,n}. \quad (2)$$

The benchmark keeps these values synthetic: absolute scale is a proxy, and object priors are public-source envelopes rather than measured signatures [8]–[13].

A. Radar Simulation Best Practices

The simulation follows standard radar-review practice while keeping the claim boundary explicit. Waveform assump-

tions expose carrier frequency, bandwidth, CPI, PRF/CRF, pulse/chirp count, range span, range resolution, Doppler resolution, and velocity resolution. The range-Doppler proxy uses

$$\Delta R = \frac{c}{2B}, \quad \Delta f_D = \frac{1}{T_{\text{CPI}}}, \quad \Delta v = \frac{\lambda}{2T_{\text{CPI}}}, \quad (3)$$

with approximate unambiguous velocity $\lambda f_{\text{PRF}}/4$ for branch cards. Public-proxy target modeling includes fixed-wing pusher-prop geometry, aspect RCS uncertainty, Swerling-like fluctuation, scenario kinematics, and a broad 150–217 Hz pusher-prop micro-Doppler prior. Hard negatives include birds and RC fixed-wing with overlapping cadence or aspect. Environment and receiver stress include Weibull/K-like clutter, thermal noise buckets, phase noise, AGC compression, ADC quantization, dropped CPI, Doppler folding, calibration offset, RFI bursts, and multipath masking [14]–[17].

Data-processing path from synthetic scenario to holdout KPI

Group split, detector schema, EI lock, and metric basis are visible as evidence artifacts.

Path: Scenario groups -> Synthetic sensing -> Detector-view schema -> Train/CV branch -> Blind holdout.

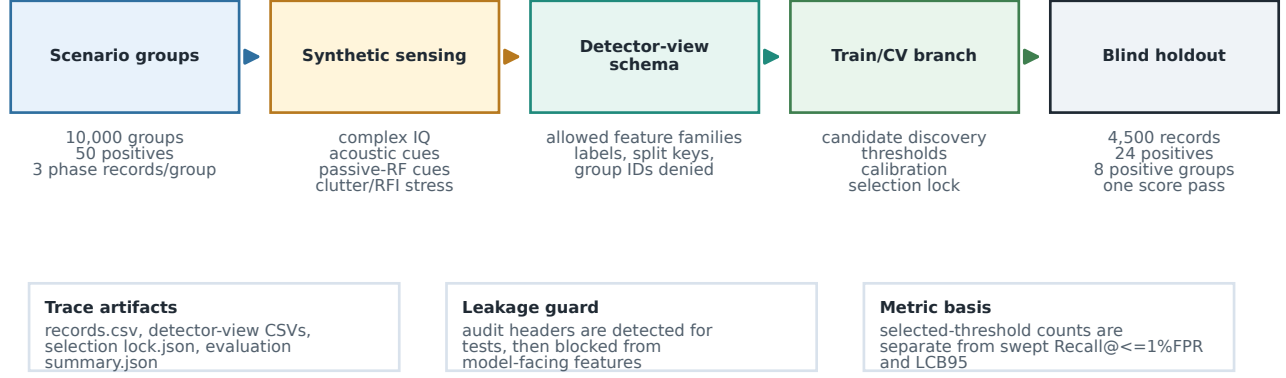


Fig. 2. End-to-end data-processing path. Scenario groups and phase records generate synthetic radar/cue products; detector-view materialization enforces feature denylisting; train/CV rows drive candidate discovery, thresholds, and monotone calibration; and the blind holdout is scored once after selection lock.

TABLE II
DATA PROCESSING TRACEABILITY CHECKLIST

Stage	Produced artifact	Reviewer check	Failure mode guarded
Scenario sampling	scenario_manifest.csv, dataset_manifest.json	group IDs and split roles exist before phase expansion	row-level leakage through re- peated phases
Synthetic sensing	records.csv, raw IQ refer- ences, acoustic and passive-RF cue summaries	generated radar/cue products inherit public-proxy priors and group split	undocumented signal-source mixing
Detector views	detector-view CSVs and detector_view_schema_evidence.csv	labels, group IDs, split keys, audit headers, and generator metadata are blocked	shortcut learning from audit fields
Train/CV branch	selection_lock.json, component weights, thresholds, calibrators	candidate search, weighting, thresh- olding, and calibration are train/CV- only	holdout tuning or silent rese- lection
Blind holdout	evaluation_summary.json, main_kpi_gain_table.csv	selected-threshold counts, swept Recall@≤1%FPR, and LCB95 are separate fields	conflating locked-threshold counts with swept low-FPR recall

B. Monte Carlo Scenario Construction

The main run consists of 10,000 scenario groups, 50 positive groups, and 30,000 phase records generated from one seed. Each group receives site, range, aspect, weather/noise, hard-negative family, and positive/negative role, then expands into three locked windows: initial take-up, climb transition, and cruise altitude. Split assignment occurs at the scenario-group level so all phase records from one group share the same split role and fold. The final holdout contains 1,500 groups with eight positive groups and 24 positive phase records; the train/CV branch contains the remaining 42 positive groups. The evidence bundle emits monte_carlo_setup_rows.csv to make the randomization and group lock inspectable.

IV. DETECTOR VIEWS AND HUMAN BASELINES

The raw data product is a synthetic intermediate: raw IQ references, range-Doppler diagnostics, acoustic cue summaries, passive-RF cue summaries, and scenario records. The detector-processing contract reduces those products to branch-level views. Model-facing matrices receive score summaries and cue features; labels, split keys, group IDs, audit headers, time locks, and restricted generator state are blocked by the feature denylist. This is the Detector-View Modeling Contract carried by detector_view_schema_evidence.csv.

Detector archetypes are public-source role envelopes. The X/Ku C-UAS branch is anchored by public descriptions of low-slow-small radar behavior such as Blighter A400 and KuRFS [19], [20]. The tactical S-band branch follows public MHR/nMHR-style hemispheric surveillance roles [21]. The

TABLE III
RADAR MODEL CARD AND DERIVED RESOLUTION ASSUMPTIONS

Branch	f_0 GHz	BW MHz	PRF/CRF Hz	CPI ms	Chirps	Span m	ΔR m	Δv m/s
X/Ku C-UAS	10.0	600	4000/4000	6.0	24	5.0	0.250	2.498
S-band AESA/MHR	3.1	180	4000/4000	6.0	24	16.7	0.833	8.059
GBAD 3D/4D	10.3	300	4000/4000	6.0	24	10.0	0.500	2.426

Scenario balance and leakage diagnostics

Scenario groups are locked before phase expansion; the evidence keeps label, site, range, aspect, noise, and hard-negative family counts together.

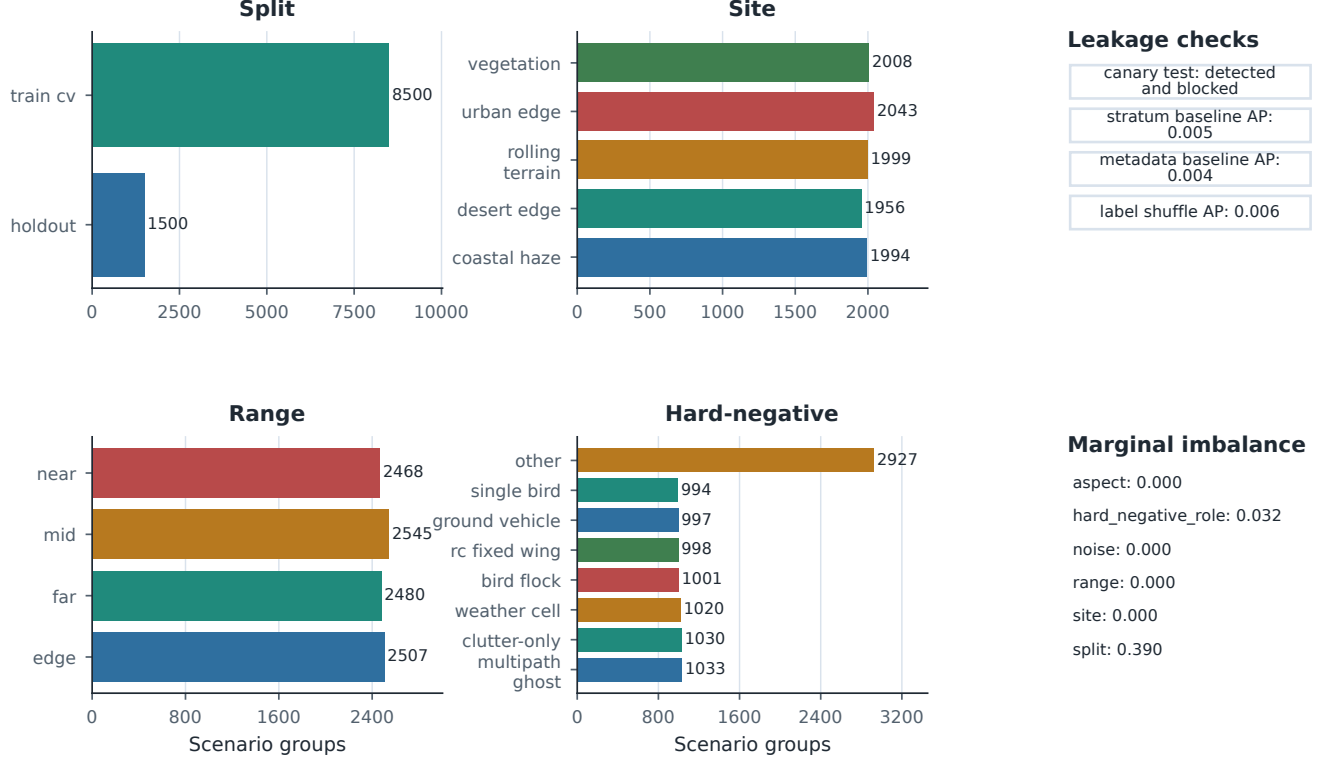


Fig. 3. Scenario balance and leakage diagnostics. Scenario groups are locked before phase expansion, and the evidence bundle keeps split, label, site, range, aspect, noise, and hard-negative family counts together with canary and sanity checks. Audit-only split/group headers are reported as detected and blocked by the model-facing schema rather than as detector features.

GBAD cueing branch follows public wide-area 3D/4D radar roles such as Giraffe 1X and Sentinel summaries [22], [23]. Public role envelope means plausible branch role and observable family, not sensitivity, ECCM, exact Pd/Pfa, or military performance.

V. SENSOR FUSION BASELINE

Low-altitude UAS detection is rarely a single-detector problem. Radar branches see range-Doppler and track structure; acoustic cues see cadence and cross-node agreement; passive-RF cues see absence, contamination, and provenance. Classical multisensor fusion, tracking/data association, and multi-source uncertainty practice motivate an accepted fusion comparator before EI is introduced [24]–[27]. The accepted prior fusion lane combines branch evidence while preserving human-designed thresholding and calibration. EI is judged against this fusion lane, not against a deliberately weak single detector.

VI. ENGINEERED INTELLIGENCE

EI is implemented as auditable algorithm discovery rather than as an opaque model handle. It decomposes the problem into candidate score families, evolves and ranks sparse nonnegative combinations on train/CV rows, fits a monotone odds calibrator, writes a selection lock, and then scores holdout once. The source is observable; the evidence bundle records source hashes, component weights, trace rows, and the no-holdout-selection policy. This framing is aligned with program-search and agentic scientific-discovery work, while keeping the benchmark claim narrow and reproducible [28]–[31].

A. Evolution Trace and Locked Holdout Endpoint

The most important EI audit artifact is the search trace, not only the final score. The advanced lane writes `evolution_trace.csv` with candidate order, stage,

TABLE IV
DETECTOR-PROCESSING BASELINE LADDER

Lane	Processing idea	Role in comparison
X/Ku radar	CFAR-style range-Doppler concentration, micro-Doppler spread, clutter stress	High-resolution radar branch sanity check [2], [3], [15]
S-band radar	Coarser MTD/range-velocity summaries and track confidence	Tactical radar branch stress check [2], [16], [18]
GBAD cueing	Horizon/revisit/stale-track cue score	Wide-area cueing branch check
Acoustic cue	Spectral cadence, amplitude stability, node agreement	Propulsion-cadence supporting cue
Passive RF	No-signal, RFI burst, provenance quality, sparse RF cue geometry	Missingness and multimodal provenance stress
Prior ML	Tabular and sequence learners on denylisted detector views	Ordinary learned controls under the same split
Accepted fusion	Human-engineered layered fusion with train/CV thresholding	Main accepted-practice comparator for EI

Phase method ladder: detectors -> accepted fusion -> EI

Branch rows use holdout AP when fixed-FPR recall is unavailable; fusion and EI use the low-FPR operating metric.

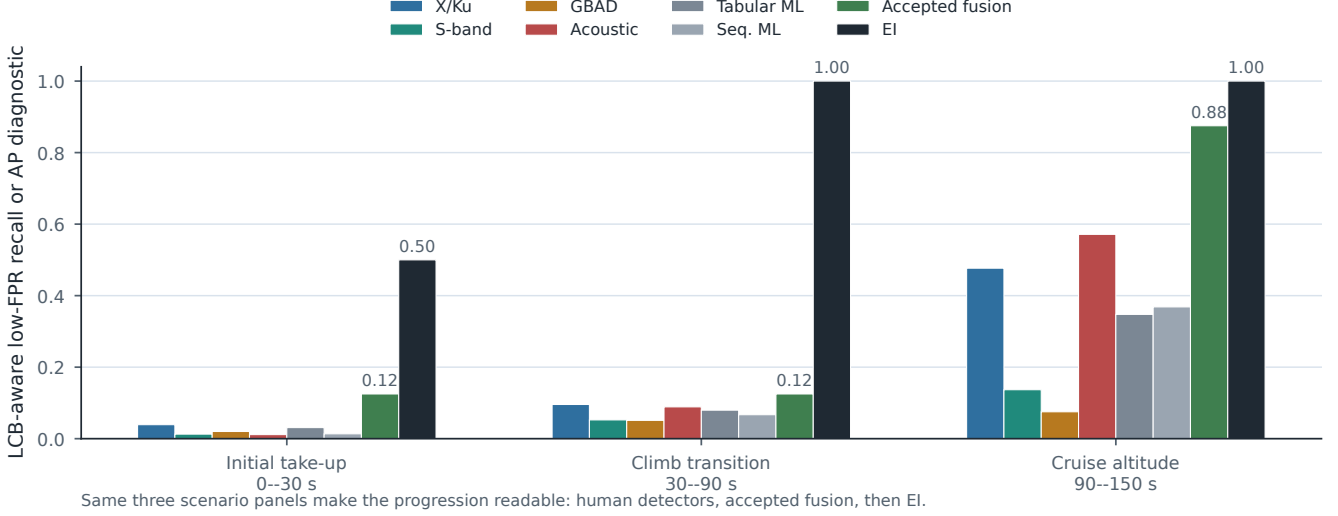


Fig. 4. Phase-wise method ladder. The same three scenario panels show progression from human detector branches to accepted fusion and then to EI. The plotted operating metric is Recall@ $\leq 1\%$ FPR when available, with AP retained as a secondary diagnostic for branch rows.

family, calibrator, feature/component count, train/CV objective, train/CV AP, train/CV ROC AUC, running-best values, and exactly one CV-chosen row. The paper evidence normalizer emits `ei_evolution_trace.csv` and `ei_evolution_summary.json`. Every trace row has `selection_split=train_cv` and `holdout_rows_used_for_selection=0`.

This separation is essential. A curve of holdout scores across candidate attempts would invite holdout shopping unless each point came from a separately pre-declared lock. EchoForge therefore reports the search trajectory as train/CV evidence and shows the holdout only once, as the endpoint of `selection_lock.json`. The holdout is not used to draw the evolution curve.

VII. EVALUATION PROTOCOL

Let s_i be a detector score. For EI, the fused score is a nonnegative weighted sum over component scores,

$$z_i = \sum_{j=1}^k w_j s_{ij}, \quad w_j \geq 0, \quad \sum_{j=1}^k w_j = 1. \quad (4)$$

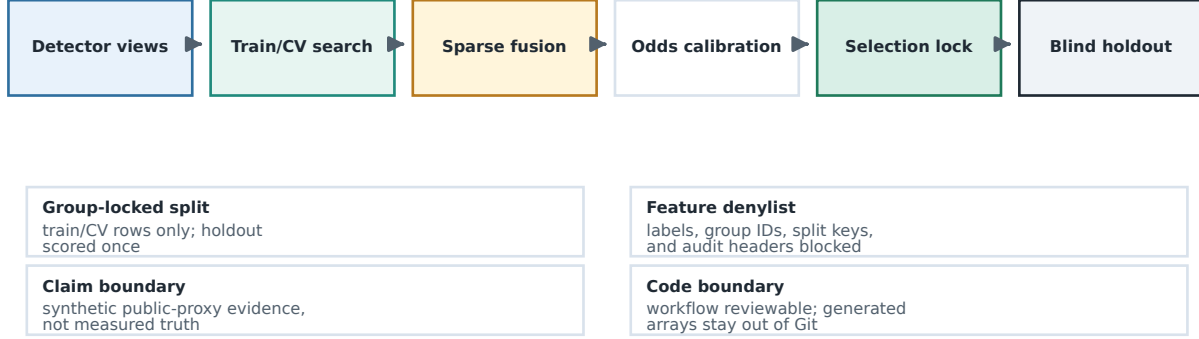
The paper-facing calibration step is a monotone geodesic-odds transform,

$$\hat{p}_i = \sigma \left(a \log \frac{z_i + \epsilon}{1 - z_i + \epsilon} + b \right), \quad (5)$$

where σ is the logistic function. Thresholds and calibrators are fit on train/CV rows only. The evaluation metrics are ROC AUC, AP, Recall@ $\leq 1\%$ FPR, accuracy, precision, recall, FPR, F1, Brier score, and ECE. Group-block bootstrap confidence intervals resample scenario groups, not phase rows, which matches the locked group structure [32]–[35]. Calibration and classifier diagnostics follow standard practice [36]–[44].

Engineered Intelligence workflow and audit boundaries

What EI does: train/CV candidate discovery -> sparse fusion -> calibration -> locked holdout score



Raw component handles, selection_lock.json, and component CSVs remain evidence artifacts; reader-facing labels summarize their role.

Fig. 5. Engineered Intelligence workflow and audit boundaries. Detector-view tables, train/CV-only candidate discovery, sparse nonnegative fusion, geodesic-odds calibration, EI selection lock, and blind holdout scoring are separated from group-lock, feature-denylist, claim-boundary, and code-disclosure rails.

TABLE V
EI CANDIDATE DISCOVERY AND LOCKED SCORING ALGORITHM

Step	Operation
1	Read train/CV detector-view scores after the denylist removes labels, split fields, group IDs, time locks, and audit headers.
2	Generate candidate component families from radar, passive-RF, acoustic, transport/geometry, and fusion-context score views.
3	Optimize sparse nonnegative component weights on train/CV rows only.
4	Fit a monotone geodesic-odds calibrator and threshold policy without holdout rows.
5	Write <code>selection_lock.json</code> and freeze component IDs, weights, calibrator, and threshold source.
6	Score the blind holdout once and emit KPI, calibration, false-alarm, and leakage evidence.

A. Primary KPI

The headline KPI is the lower 95% group-block bootstrap bound of recall at $\text{FPR} \leq 1\%$, abbreviated LCB95 Recall@ $\leq 1\%\text{FPR}$. AP, ROC AUC, raw recall, precision, F1, calibration, and false-alarm breakdowns are guardrails. Three quantities are kept separate: selected-threshold TP/F-P/FN counts use the threshold fixed by train/CV or the EI lock; Recall@ $\leq 1\%\text{FPR}$ is a swept holdout operating-point diagnostic; and LCB95 is the 2.5th percentile of the group-block bootstrap distribution for that swept low-FPR recall.

VIII. RESULTS

The gain row is the direct result: EI raises the primary LCB95 metric by +0.630, raises point Recall@ $\leq 1\%\text{FPR}$ by +0.542, and reduces selected-threshold false positives from 49 to 3 while conceding ROC AUC. Because the positive holdout

has $n = 8$ groups, phase and family conclusions are diagnostic rather than stable field-rate estimates.

IX. DIAGNOSTICS

The leakage and realism diagnostics include audit-header detection plus model denylisting, label-shuffle sanity, stratum-only and metadata-only baselines, and nearest-neighbor train/holdout scans. The compare-only KTH overlay is useful for distribution-shape checks, not positive-class validation. The comparable view controls also expose a useful critic's point: passive-RF-only ranking can be strong on AP and swept low-FPR recall, but its selected-threshold calibration and lock status differ from the full EI artifact. The paper therefore reports EI as the pre-specified calibrated selected-threshold artifact, not as a universal dominator of every view control.

X. LIMITATIONS

The most important limitation is also the simplest: a synthetic benchmark is not measured truth. The present corpus should be read as public-proxy evidence with declared priors, not as a fielded sensor surrogate. The comparison against KTH and other public anchors is compare-only and insufficient to justify named-platform claims. That boundary follows the usual verification-and-validation distinction in computational science and reproducibility recommendations for machine-learning research [45]–[48].

The statistical limitation is central. The corpus is a single-seed run with 50 positive groups total and only eight positive groups in the holdout split. Group-block bootstrap intervals are necessary but not sufficient for broad claims. The physics limitation is also explicit: branch cards use simplified public-proxy RCS, clutter, weather, RFI, waveform, and processing assumptions. The paper contains no hardware-in-the-loop evidence, no measured positive-class validation, no track-level FAR, and no deployment-performance claim. The next

EI evolution money plot: train/CV search plus one locked holdout endpoint

The curve is internal train/CV evidence only; the holdout appears once after selection lock.

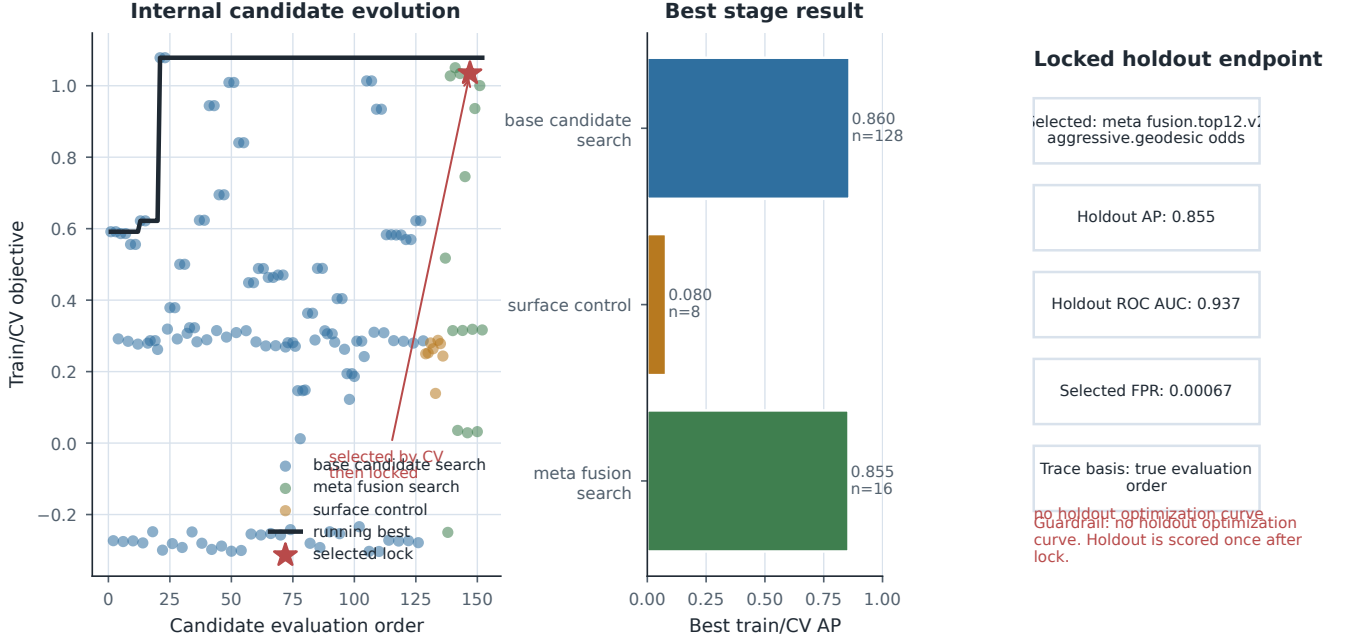


Fig. 6. EI evolution money plot. Left: internal train/CV candidate evaluations in actual evaluation order with the running best shown as a bold line. Middle: best train/CV AP reached by each search stage. Right: the single locked holdout endpoint after the selection lock. The holdout is not used to draw the evolution curve.

TABLE VI
EI EVOLUTION TRACE AUDIT CONTRACT

Artifact	Required fields	Reviewer check	Failure mode guarded
evolution_trace.csv	candidate order, stage, family, calibrator, train/CV metrics, running best	actual search trace rather than sorted leaderboard	fake evolution plot
selection_lock.json	frozen component IDs, weights, calibrator, threshold source, no-holdout policy	trace row matches the locked artifact	silent reselection
ei_evolution_summary.json	trace basis, selected row, one holdout endpoint	strict mode fails on leaderboard-only fallback	rank-order masquerading as evolution
Holdout endpoint	AP, ROC AUC, FPR, F1, gate status for the locked artifact only	holdout appears once after lock	holdout tuning

TABLE VII
MAIN KPI GAIN LEDGER VERSUS ACCEPTED PRIOR FUSION

Metric	Prior	EI	Change
LCB95	0.083	0.713	+760%, 7.60x
Recall@ $\leq 1\%$ FPR			
Point	0.292	0.833	+185.7%, 2.85x
Recall@ $\leq 1\%$ FPR			
Average precision	0.128	0.830	about +547%
Selected-threshold FP	49	3	93.9% fewer
F1	0.241	0.826	+243%
ROC AUC	0.938	0.916	guardrail trade-off

useful steps are multi-seed corpora, richer site/weather/RFI stressors, larger positive holdouts, hardware-in-the-loop hooks,

and finer-grained uncertainty reporting.

XI. CONCLUSION

EchoForge demonstrates a strict-open path for low-altitude UAS radar and multimodal sensing studies: public-proxy priors, deterministic generation, detector-view denylisting, group-locked holdout evaluation, group-block uncertainty estimates, compare-only measured-anchor checks, reproducible detector/fusion benchmarking, and transparent EI-discovered algorithmic improvement. The headline result is LCB95 Recall@ $\leq 1\%$ FPR of 0.713 for EI versus 0.083 for accepted prior fusion, a **+760%** relative gain, with point Recall@ $\leq 1\%$ FPR of 0.833 versus 0.292. These are synthetic public-proxy results under declared assumptions. They are useful for review, robustness work, and reproducible comparison, but they are not measured truth or operational performance claims.

Holdout KPI: EI vs accepted prior fusion

Primary KPI gain vs best practice: +759% LCB95; +185% point recall; selected FP 49->3

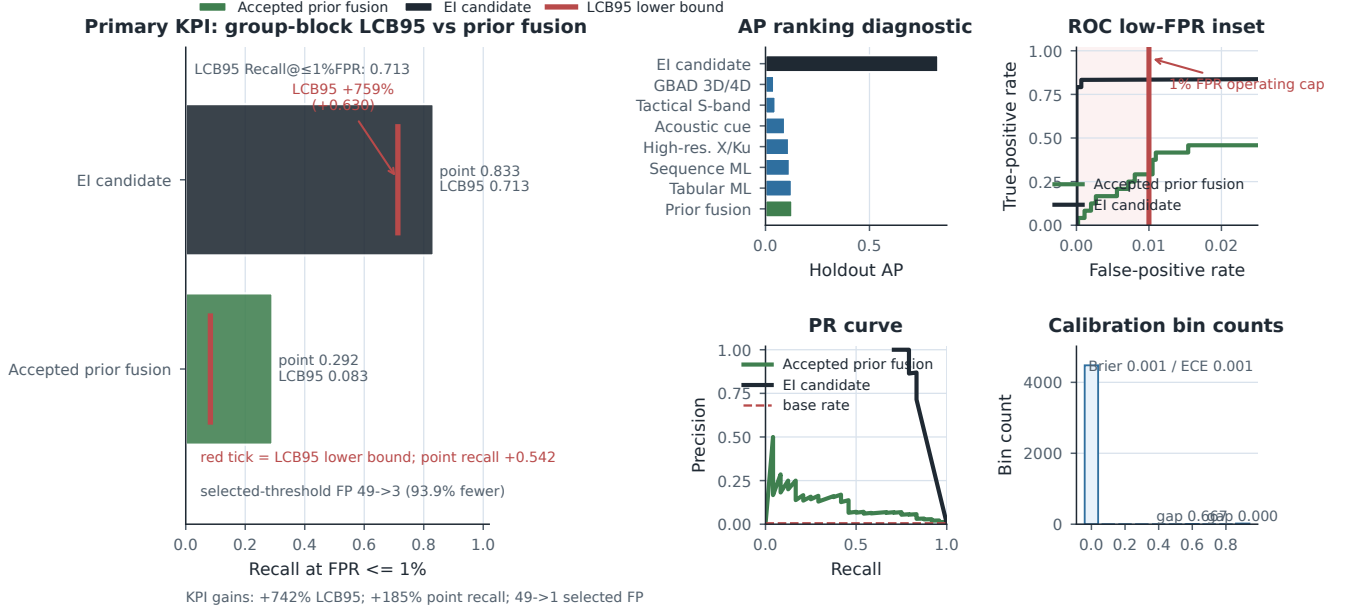


Fig. 7. Holdout primary KPI, ranking diagnostics, low-FPR ROC/PR behavior, and calibration. Red vertical ticks are group-block LCB95 lower bounds, and the pale red shaded zone with the thick red line marks the 1% FPR operating cap. Recall@ $\leq 1\%$ FPR and its LCB95 are the headline quantities; AP, ROC AUC, PR shape, and calibration bins are supporting diagnostics.

TABLE VIII
 HOLDOUT SUMMARY SEPARATING RANKING, SELECTED-THRESHOLD COUNTS, AND SWEEPED FIXED-FPR METRICS

Method	AP	ROC AUC	F1	ECE	Threshold source	TP/FP/FN	Sel. FPR	Recall@ $\leq 1\%$ FPR	LCB95
accepted prior fusion	0.128	0.938	0.241	0.016	train/CV max-F1	10/49/14	0.010947	0.292	0.083
EI	0.830	0.916	0.826	0.002	EI selection lock	19/3/5	0.000670	0.833	0.713
EI gain vs prior	+0.702	-0.021	+0.585	-0.014	locked vs train/CV	+9/-46/-9	-0.010277	+0.542	+0.630

Phase behavior and false-alarm burden

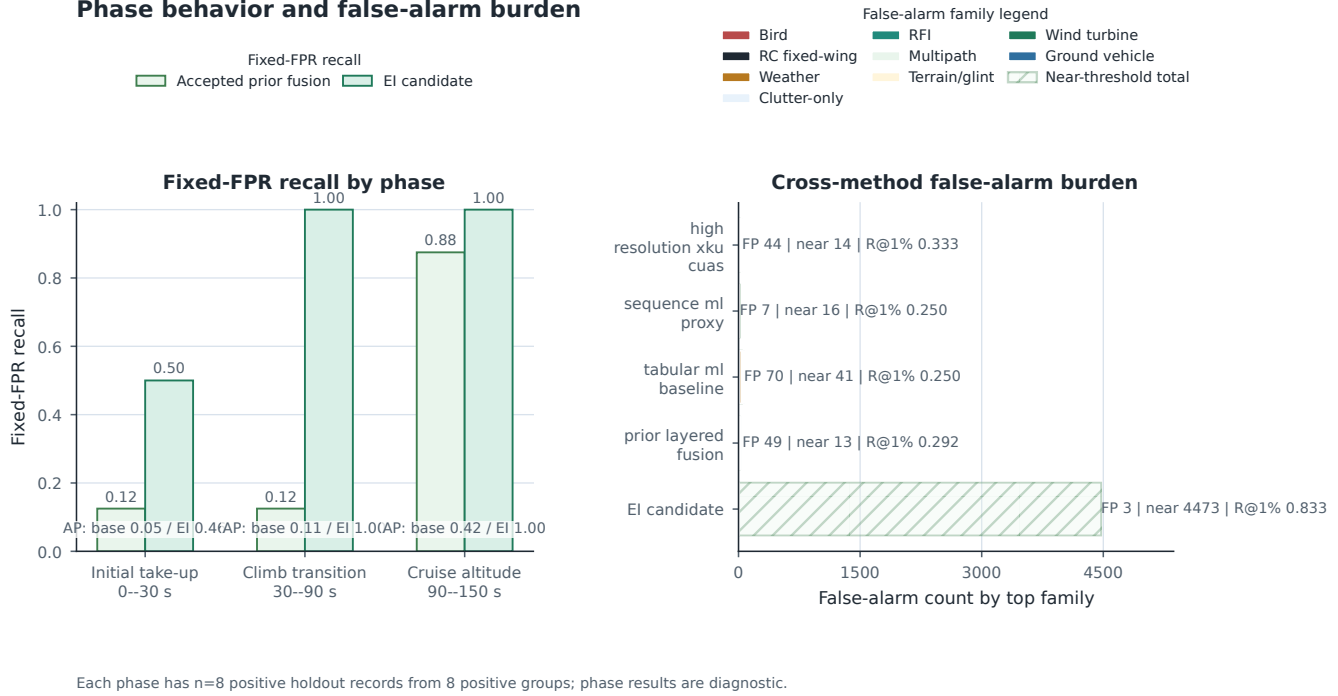


Fig. 8. Per-phase fixed-FPR recall and cross-method false-positive burden. The left panel keeps the operating metric simple: bars are Recall@ $\leq 1\%$ FPR for accepted prior fusion and EI, while AP is printed only as diagnostic text. The right-panel false-alarm family legend maps stacked colors to hard-negative families; pale/hatched bars show near-threshold totals, and the FP, near, and R@1% labels are diagnostic robustness context. Each phase has only eight positive holdout records from eight positive groups, so phase conclusions are diagnostic.

EI components and comparable holdout deltas

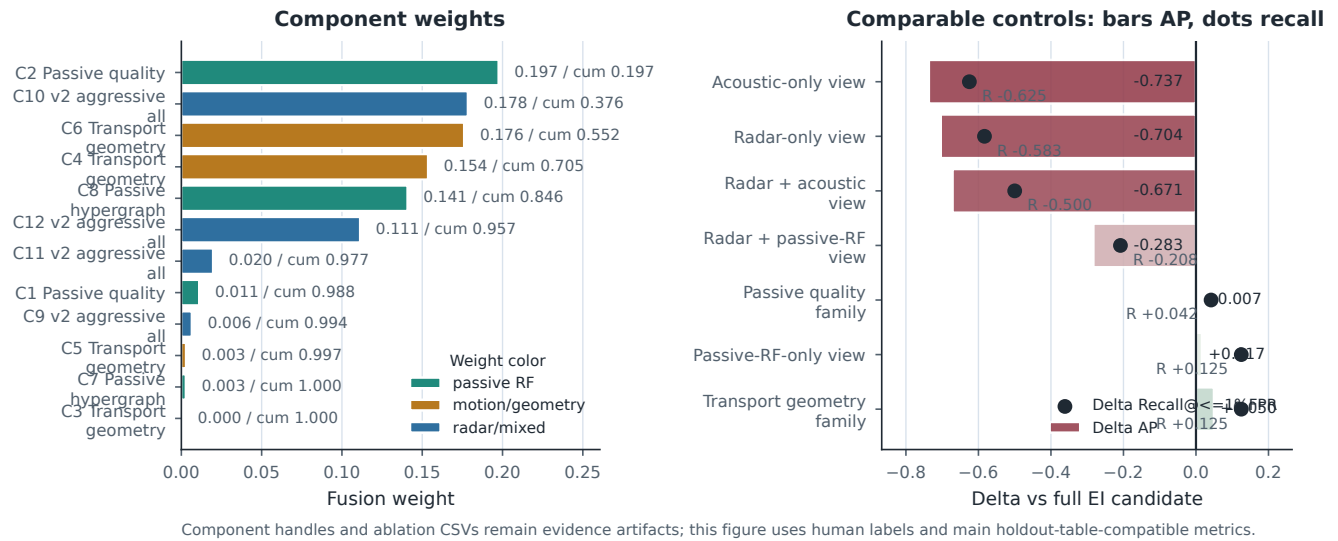


Fig. 9. EI component weights and comparable holdout controls. The left panel sorts human-readable component weights; the right panel reports AP and fixed-FPR recall deltas computed with the same main holdout-table-compatible metric definitions as Table VIII. Raw component handles and ablation-internal diagnostics remain in evidence artifacts.

Radar model card, resolution budget, and public-proxy boundary

Branch	Band	BW MHz	dR m	PRF/CRF Hz	CPI ms	Pulses	dfD Hz	dv m/s
High-res. X/Ku	X/Ku	600	0.250	4000/4000	6.0	24	166.7	2.498
Tactical S-band	S	180	0.833	4000/4000	6.0	24	166.7	8.059
GBAD 3D/4D	X/Ku cue	300	0.500	4000/4000	6.0	24	166.7	2.426

Public-proxy assumptions

positive class: fixed-wing pusher-prop
public proxy
geometry: swept/delta wing; rear pusher
propulsor
speed: 45--60 m/s; prop micro-Doppler:
150--217 Hz
phase windows: 0--30 s, 30--90 s, 90--150 s
launch proxy: rail/catapult take-up
RCS/aspect: -28 to -2 dBsm public-source
proxy

Receiver impairments

AGC
compression
clock drift
quantization
dropped CPI
PRF ambiguity
Doppler folding
calibration offset
multipath masking

Cue definitions

acoustic: node-wise spectral cadence, cross-node agreement, and amplitude stability
passive RF: no-signal / RFI burst / clock-offset / provenance-quality geometry
claim boundary: public-proxy assumptions only

All values are public-proxy or synthetic archetype assumptions; no measured-platform truth is implied.

Fig. 10. Radar model card and signal-chain parameter panel. The figure surfaces the public-proxy assumptions behind the benchmark so detector results can be read against explicit carrier, bandwidth, PRF/CPI, and cue definitions.

Range-Doppler proxy diagnostics for positive and hard-negative samples

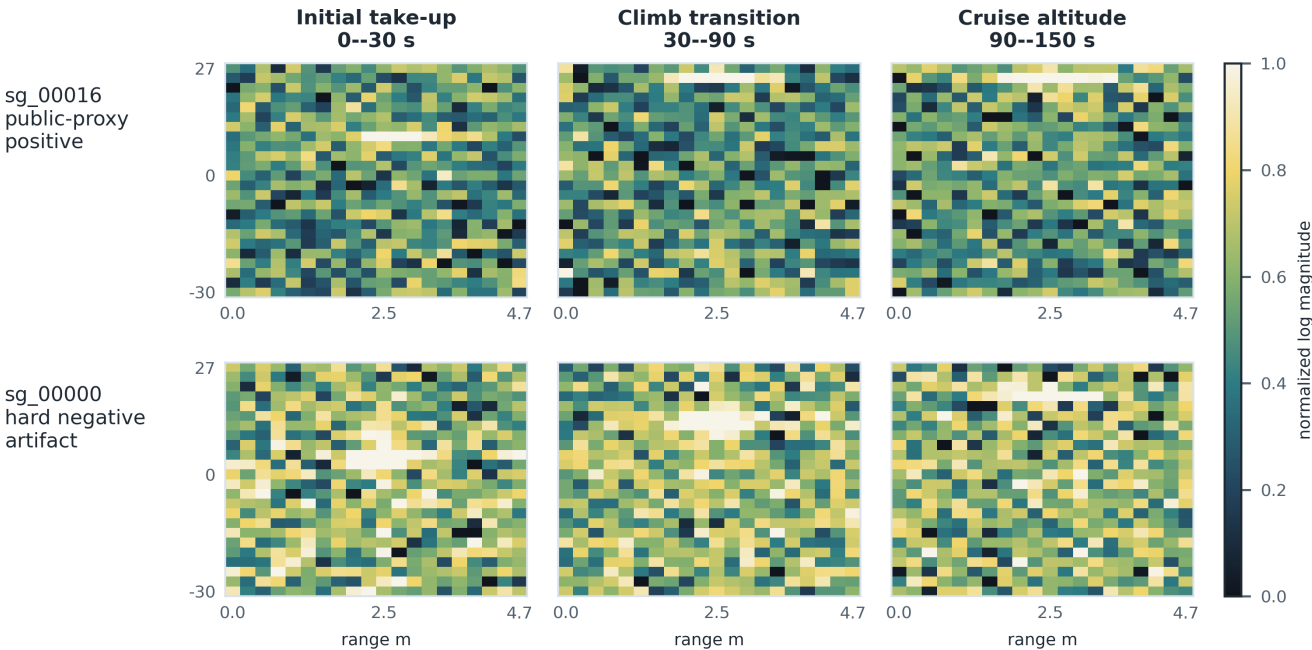


Fig. 11. Qualitative range-Doppler proxy sanity panels for a positive public-proxy group and a hard-negative group. Axes show relative range and velocity-equivalent Doppler bins; color is normalized log-magnitude. These panels are qualitative diagnostics, not measured imagery and not detector input for the main KPI.

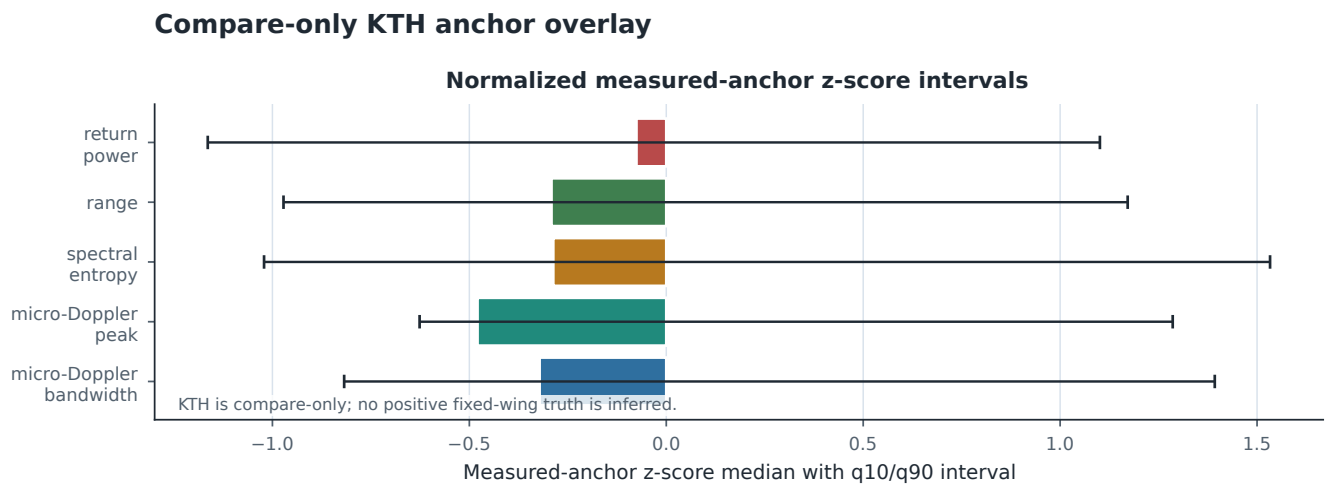


Fig. 12. Compare-only KTH anchor overlay. Features are normalized to measured-anchor z-scores before plotting, so the figure compares distribution shape rather than mixing raw physical units.

Appendix modeling map: positive proxy, environment, and detector views

Review surface only: assumptions are auditable without expanding the measured-truth claim.



Evidence rows: public_proxy_model_detail_rows.csv, environment_impairment_model_rows.csv, detector_view_schema_evidence.csv

Fig. 13. Appendix modeling map for the positive proxy, environment/receiver stress, and detector views. This figure is a review surface: it shows how assumptions are separated from prohibited measured-truth claims.

TABLE IX
FIXED-WING PUSHER-PROP / IRANIAN-DRONE PUBLIC-PROXY MODELING CARD

Modeling item	Public-proxy assumption	Radar consequence	Explicit boundary
Airframe geometry	Delta or swept fixed wing, rear pusher propulsor, approximate 3.3–3.7 m length, 2.3–2.7 m span, 0.35–0.75 m height	Aspect-dependent projected area and RCS envelope	Public visual/summary proxy only; no measured Iranian-platform signature
Launch/take-up	Rail/catapult take-up proxy with short booster-assist stress in the first 0–30 s	Initial phase has ground coupling, multipath, and less settled kinematics	No route, launch-site, operational, or evasion model
Kinematics	45–60 m/s speed envelope across take-up, climb, and cruise windows	Doppler centroid and track-rate priors for branch summaries	Broad public kinematic envelope only
Propulsion Doppler	micro-Pusher-prop cadence proxy of 150–217 Hz	Narrow high-cadence modulation family distinct from many bird wingbeat bands but overlapping RC prop cadence	Not a measured micro-Doppler signature
RCS/aspect	-28 to -2 dBsm aspect envelope with Swerling-like fluctuation proxy	Controls per-branch detectability and hard-negative overlap	Synthetic stress envelope; no sensor-specific Pd/Pfa claim
Phase labels	Initial take-up 0–30 s, climb transition 30–90 s, cruise altitude 90–150 s	Phase-stratified Recall@ $\leq 1\%$ FPR, and alarm diagnostics	AP, false-Phase conclusions are diagnostic because the holdout has eight positive groups

APPENDIX A MODELING, DETECTOR, AND SOURCE APPENDIX

The appendix ledger separates what each source family is used for from what it is not allowed to prove. This is the reviewer-facing boundary for the public-proxy positive class, hard negatives, detector archetypes, and compare-only measured anchors.

A. Rich Public-Proxy Modeling Appendix

This appendix states how the benchmark models the positive public-source family, the radar/noise environment, the auxiliary cue families, and hard negatives. The public-source positive class is inspired by open reporting on Iranian fixed-wing pusher-prop loitering-munition families, but EchoForge does *not* claim a measured Iranian-drone radar signature, route model, payload model, or operational performance surrogate. The paper-facing class remains the fixed-wing pusher-prop public proxy.

TABLE X
NOISE, CLUTTER, RFI, AND RECEIVER MODELING DETAILS

Family	What is generated	Why it matters for radar review	Boundary
Thermal/SNR stress	Synthetic SNR/CNR buckets attached to radar branches and detector-view features	Prevents a trivial clean-signal benchmark	Not a calibrated receiver-noise figure for hardware
Weibull/K-like clutter	Terrain, sea, vegetation, and mixed-scene clutter stressors	Covers non-Gaussian clutter regimes used in radar detection literature	Site-measured clutter truth is not claimed
Weather and sea clutter	Rain/dust/weather-cell, coastal haze, and sea/terrain glint families	Creates near-threshold false-track pressure	Not a meteorological rate model
Multipath ghosts	Synthetic ghost tracks and reflection-like interference	Tests whether fusion overreacts to plausible false tracks	No deployment geometry or site-specific ray tracing
RFI bursts and passive-RF missingness	Sparse RF cues, no-signal behavior, burst contamination, clock-offset, and provenance quality	Tests multimodal fusion when RF is absent, sparse, or contaminated	No emitter library, payload command, operator, or exploitation claim
Receiver impairment	AGC compression, clock drift, quantization, dropped CPI, Doppler folding, and calibration offset	Forces detector views to tolerate sensor-like imperfections	Simplified impairment families, not hardware qualification

TABLE XI
SYNTHETIC RADAR SIMULATION PROCESSING DETAIL

Processing step	Modeled quantity	How it becomes evidence	Red-line boundary
Target return	aspect-dependent public-proxy RCS, phase-stable complex amplitude, range migration, Doppler centroid	IQ magnitude diagnostics and detector features such as concentration and Doppler spread	no measured platform signature
Micro-Doppler	pusher-prop 150–217 Hz proxy plus bird/RC cadence stressors	branch summaries and hard-negative false-alarm interpretation	cadence bands are broad priors, not identity labels
Clutter/noise	thermal/SNR buckets, Weibull/K-like clutter, sea/weather/terrain glint, vegetation motion	score degradation and near-threshold hard-negative pressure	no site-measured clutter truth
Receiver effects	AGC compression, drift, quantization, dropped CPI, Doppler folding, calibration offset	detector-view robustness features and failure slices	no hardware qualification
Cue streams	acoustic cadence/agreement and passive-RF no-signal/RFI/provenance geometry	non-radar branch scores and EI component inputs	no emitter library or operator inference
Feature contract	denylisted schema converts generated products to detector views	train/CV and holdout scores, leakage diagnostics, and KPI tables	labels, split keys, group IDs, and generator internals blocked

The appendix is not a claim-expansion device. It is a review surface. It explains the synthetic assumptions behind the benchmark while keeping the central claim narrow: EI improves the low-FPR operating point over accepted fusion practice on this declared synthetic public-proxy run.

APPENDIX B REPRODUCIBILITY CLI APPENDIX

These commands reproduce the paper lane; they are benchmark commands, not field-deployment instructions. The generated evidence bundle also emits `cli_reproduction_commands.csv`.

Generate the synthetic scenario and sensor corpus

```
rtk python3 -m detection.generate_main_run \
  --profile fixed-wing-pusher-proxy-v2 \
  --out-root outputs/training-data/runit-fixed-wing-pusher-proxy-v2-main-run \
  --scenario-groups 10000 \
  --seed 202605210136 \
  --force
```

TABLE XII
DETECTOR-VIEW MODELING CONTRACT

View	Model-facing features	What it can support	What it cannot support
X/Ku radar	Range-Doppler concentration, micro-Doppler spread, CFAR/MTD-style scores, clutter stress	High-resolution radar branch comparison	Proprietary sensitivity, ECCM, or exact vendor Pd/Pfa
S-band radar	Coarser velocity/range summaries, track confidence, multipath stress	Tactical branch stress and cue comparison	Any claimed match to a fielded AESA/MHR implementation
GBAD cueing	Cue confidence, horizon/revisit stress, stale-track state	Wide-area cueing behavior under a public role envelope	Engagement-quality behavior or operator workflow
Acoustic	Spectral cadence, node agreement, weather/traffic stress	Supporting propulsion-cadence cue	Exact microphone layout or field localization performance
Passive RF	No-signal, RFI burst, sparse RF cue, provenance-quality geometry	RF silence/missingness and provenance-quality stress	Emitter identification, command links, payload inference, or operator identity
Fusion	Source confidence, stale-track penalty, cross-sensor confirmation, false-track accounting	Accepted-practice comparator and EI input context	Operational C2 equivalence

TABLE XIII
ABLATION AND APPENDIX ARTIFACT MAP

Artifact	Contents	Reviewer question answered
data_processing_trace_Stage-level	scenario-to-holdout trace, including input artifacts, output artifacts, reviewer checks, and claim boundaries	Can I reconstruct the processing path without reading generator internals?
main_kpi_gain_table.csv	Prior-fusion and EI values, absolute changes, relative percentages, and gain/reduction direction	What exactly is the +760% LCB95, +185.7% point-recall, and 49-to-3 false-positive claim computed from?
public_proxy_model_detector_classification	Positive-class geometry, kinematics, RCS, micro-Doppler, launch, phase, and prohibited-inference rows	How is the Iranian-drone public proxy represented without claiming measured truth?
environment_impairment_Clutter, noise, weather	Clutter, noise, weather, RFI, multipath, receiver impairment, and hard-negative stress rows	What noise and artifact regimes are actually modeled?
detector_view_schema_eAllowed feature	families and denied audit-only fields for every detector view	Are labels, split keys, group IDs, or generator state leaking into models?
false_alarm_by_method_Family-wise	false alarms for EI and the strongest comparator methods	Which hard negatives generate false alarms and what do the colors in Fig. 8 mean?
selected_component_humReader-facing	EI component weights, modalities, calibrators, and cumulative weights	What does EI fuse at a high level?

Run accepted detector and fusion baselines

```
rtk python3 -m detection.run_main_run_detectors \
--data-root outputs/training-data/runit-fixed-wing-pusher-proxy-v2-main-run \
--out-root outputs/detection/runit-fixed-wing-pusher-proxy-v2-main-run \
--folds 5 \
--seed 202605210136 \
--force
```

Run EI search, selection lock, and holdout scoring

```
rtk python3 -m detection.run_advanced_main_run_detectors \
--data-root outputs/training-data/runit-fixed-wing-pusher-proxy-v2-main-run \
--out-root outputs/detection/runit-fixed-wing-pusher-proxy-v2-main-run-advanced-evolution \
--folds 5 \
--seed 202605210136 \
--search-profile v2_aggressive \
```

```
--candidate-limit 128 \
--evolution-rounds 5 \
--evolution-sample-rows 4500 \
--write-component-scores \
--force
```

Build evidence, source appendix, figures, and PDF

```
rtk python3 -m detection.paper_evidence --strict-ei-trace --force
rtk python3 paper/generate_source_appendix.py --strict
rtk python3 paper/generate_figures.py --strict
rtk bash paper/build.sh --copy-tracked
rtk python3 paper/validate_paper.py \
--tex paper/echoforge_ieee.tex \
--bib paper/references.bib \
--pdf target/paper/echoforge_ieee.pdf \
--figures-dir paper/figures \
--paper-evidence-root outputs/paper-evidence/tier1-final
```

APPENDIX C SOURCE PROVENANCE APPENDIX

The listings below are generated from `paper/source_appendix_manifest.json`. Color key: `human-origin`, `generated/evolved-origin`, `mixed-origin`, and `redacted escrow-only`.

A. Human-Origin Radar Baseline Scores

Source: `detection/main_run_detectors.py`; origin: `human-origin`.

```
_cfar_score (lines 56–60): 0056 def _cfar_score(iq: np.ndarray) -> float:
0057 power = np.abs(iq) ** 2
0058 median = float(np.median(power))
0059 mad = float(np.median(np.abs(power - median)) + 1e-6)
0060 return float((np.max(power) - median) / mad)

_mtd_score (lines 63–65): 0063 def _mtd_score(iq: np.ndarray) -> float:
0064 doppler = np.abs(np.fft.fft(iq, axis=0)) ** 2
0065 return float(np.max(doppler) / (np.mean(doppler) + 1e-6))

_track_score (lines 68–74): 0068 def _track_score(iq: np.ndarray) -> float:
0069 power = np.abs(iq) ** 2
0070 range_power = power.mean(axis=0)
0071 pulse_power = power.mean(axis=1)
0072 contrast = float(np.max(range_power) / (np.mean(range_power) + 1e-6))
0073 continuity = float(1.0 / (1.0 + np.std(np.diff(pulse_power))))
0074 return contrast * continuity
```

B. Human-Origin Calibration and Reporting

Source: `detection/main_run_detectors.py`; origin: `human-origin`.

```
_fit_gaussian_log_odds (lines 119–143): 0119 def _fit_gaussian_log_odds(
0120 x: np.ndarray, y: np.ndarray
0121 ) -> tuple[np.ndarray, float, dict[str, Any]]:
0122 pos = x[y == 1]
0123 neg = x[y == 0]
0124 if len(pos) == 0 or len(neg) == 0:
0125 weights = np.ones(x.shape[1], dtype=np.float64) * 0.1
0126 intercept = 0.0
0127 return weights, intercept, {"status": "fallback_single_class"}
0128 pos_mean = np.mean(pos, axis=0)
0129 neg_mean = np.mean(neg, axis=0)
0130 pooled_var = np.var(x, axis=0) + 1e-3
0131 weights = (pos_mean - neg_mean) / pooled_var
```

```

0132 prior = (float(len(pos)) + 0.5) / (float(len(y)) + 1.0)
0133 midpoint = 0.5 * (pos_mean + neg_mean)
0134 intercept = math.log(prior / (1.0 - prior)) - float(np.dot(weights, midpoint))
0135 return (
0136 weights,
0137 intercept,
0138 {
0139 "status": "pass",
0140 "positive_rows": int(len(pos)),
0141 "negative_rows": int(len(neg)),
0142 },
0143 )

_calibrated_predictions (lines 321–385): 0321 def _calibrated_predictions(
0322 records: list[dict[str, str]],
0323 features: np.ndarray,
0324 *,
0325 folds: int,
0326 ) -> tuple[np.ndarray, list[dict[str, Any]], dict[str, Any]]:
0327 y = np.asarray([int(record["label_id"]) for record in records], dtype=np.int8)
0328 split = np.asarray([record["split_role"] for record in records])
0329 cv_fold = np.asarray(
0330 [-1 if record["cv_fold"] == "" else int(record["cv_fold"]) for record in records]
0331 )
0332 train_cv = split == "train_cv"
0333 holdout = split == "holdout"
0334 log_odds = np.zeros(len(records), dtype=np.float64)
0335 calibration_rows: list[dict[str, Any]] = []
0336
0337 for fold in range(folds):
0338 score_mask = train_cv & (cv_fold == fold)
0339 fit_mask = train_cv & (cv_fold != fold)
0340 weights, intercept, fit_info = _fit_gaussian_log_odds(features[fit_mask], y[fit_mask])
0341 log_odds[score_mask] = _predict_log_odds(features[score_mask], weights, intercept)
0342 calibration_rows.append(
0343 {
0344 "fold": fold,
0345 "fit_split_role": "train_cv",
0346 "scored_split_role": "train_cv",
0347 "fit_record_count": int(np.sum(fit_mask)),
0348 "scored_record_count": int(np.sum(score_mask)),
0349 "fit_positive_count": int(np.sum(y[fit_mask])),
0350 "fit_negative_count": int(np.sum(fit_mask) - np.sum(y[fit_mask])),
0351 "fit_status": fit_info["status"],
0352 }
0353 )
0354 weights, intercept, holdout_fit = _fit_gaussian_log_odds(features[train_cv], y[train_cv])
0355 log_odds[holdout] = _predict_log_odds(features[holdout], weights, intercept)
0356 calibration_rows.append(
0357 {
0358 "fold": "holdout",
0359 "fit_split_role": "train_cv",
0360 "scored_split_role": "holdout",
0361 "fit_record_count": int(np.sum(train_cv)),
0362 "scored_record_count": int(np.sum(holdout)),
0363 "fit_positive_count": int(np.sum(y[train_cv])),
0364 "fit_negative_count": int(np.sum(train_cv) - np.sum(y[train_cv])),
0365 "fit_status": holdout_fit["status"],
0366 }

```

```

0367 )
0368 manifest = {
0369 "dataset_profile": DATASET_PROFILE,
0370 "calibration_policy": "cv_only_train_cv_pool",
0371 "used_split_roles": ["train_cv"],
0372 "excluded_split_roles": ["holdout"],
0373 "fold_count": folds,
0374 "train_cv_record_count": int(np.sum(train_cv)),
0375 "holdout_record_count": int(np.sum(holdout)),
0376 "holdout_fit_record_count": 0,
0377 "feature_columns": [
0378 "high_res_cfar_loglp",
0379 "sband_mtd_loglp",
0380 "gbad_track_loglp",
0381 "acoustic_cadence_loglp",
0382 ],
0383 "status": "pass" if int(np.sum(holdout)) > 0 and int(np.sum(train_cv)) > 0 else "fail",
0384 }
0385 return log_odds, calibration_rows, manifest

```

_build_performance_reports (lines 239–318):

```

0239 def _build_performance_reports(
0240 records: list[dict[str, str]],
0241 method_scores: dict[str, np.ndarray],
0242 ) -> tuple[list[dict[str, Any]], dict[str, Any]]:
0243 y = np.asarray([int(record["label_id"]) for record in records], dtype=np.int8)
0244 splits = np.asarray([record["split_role"] for record in records])
0245 phases = np.asarray([record["phase_id"] for record in records])
0246 train_mask = splits == "train_cv"
0247 rows: list[dict[str, Any]] = []
0248 threshold_summary: dict[str, Any] = {}
0249
0250 for method, scores in method_scores.items():
0251 threshold, train_threshold_metrics = _select_threshold(y[train_mask], scores[train_mask])
0252 threshold_summary[method] = {
0253 "threshold_source": "train_cv_max_f1",
0254 "threshold": threshold,
0255 "train_cv_threshold_metrics": train_threshold_metrics,
0256 }
0257 slice_specs: list[tuple[str, str, np.ndarray]] = [
0258 ("all", "all", np.ones(len(records), dtype=bool)),
0259 ("train_cv", "all", train_mask),
0260 ("holdout", "all", splits == "holdout"),
0261 ]
0262 for phase in sorted(set(phases)):
0263 phase_mask = phases == phase
0264 slice_specs.append(("holdout", str(phase), (splits == "holdout") & phase_mask))
0265 for split_role, phase_id, mask in slice_specs:
0266 labels = y[mask]
0267 sliced_scores = scores[mask]
0268 binary = _binary_metrics(labels, sliced_scores, threshold)
0269 roc_auc = _float_or_none(_roc_auc(labels, sliced_scores))
0270 average_precision = _float_or_none(_average_precision(labels, sliced_scores))
0271 rows.append(
0272 {
0273 "method": method,
0274 "split_role": split_role,
0275 "phase_id": phase_id,
0276 "record_count": int(len(labels)),
0277 "positive_count": int(np.sum(labels == 1)),

```

```

0278 "negative_count": int(np.sum(labels == 0)),
0279 "threshold_source": "train_cv_max_f1",
0280 "threshold": _format_metric(threshold),
0281 "roc_auc": _format_metric(roc_auc),
0282 "average_precision": _format_metric(average_precision),
0283 "accuracy": _format_metric(binary["accuracy"]),
0284 "precision": _format_metric(binary["precision"]),
0285 "recall": _format_metric(binary["recall"]),
0286 "specificity": _format_metric(binary["specificity"]),
0287 "false_positive_rate": _format_metric(binary["false_positive_rate"]),
0288 "f1": _format_metric(binary["f1"]),
0289 }
0290 )
0291
0292 holdout_rows = [
0293 row for row in rows if row["split_role"] == "holdout" and row["phase_id"] == "all"
0294 ]
0295 holdout_by_method = {
0296 str(row["method"]): {
0297 "roc_auc": row["roc_auc"],
0298 "average_precision": row["average_precision"],
0299 "accuracy": row["accuracy"],
0300 "precision": row["precision"],
0301 "recall": row["recall"],
0302 "false_positive_rate": row["false_positive_rate"],
0303 "f1": row["f1"],
0304 "threshold": row["threshold"],
0305 }
0306 for row in holdout_rows
0307 }
0308 summary = {
0309 "dataset_profile": DATASET_PROFILE,
0310 "metric_policy": (
0311 "synthetic public-proxy benchmark metrics; thresholds selected on train/CV only"
0312 ),
0313 "methods": list(method_scores),
0314 "thresholds": threshold_summary,
0315 "holdout": holdout_by_method,
0316 "status": "pass" if holdout_rows else "fail",
0317 }
0318 return rows, summary

```

C. Mixed-Origin EI Candidate Structures

Source: detection/advanced_main_run_detectors.py; origin: mixed-origin.

CandidateSpec (lines 149–154): 0149 class CandidateSpec:

```

0150 candidate_id: str
0151 family: str
0152 subset_name: str
0153 head: str
0154 feature_indices: tuple[int, ...]

```

MetaFusionSpec (lines 158–163): 0158 class MetaFusionSpec:

```

0159 candidate_id: str
0160 component_ids: tuple[str, ...]
0161 weights: tuple[float, ...]
0162 optimizer: str
0163 top_k: int

```

D. Generated/Evolved-Origin EI Feature Lifts

Source: detection/advanced_main_run_detectors.py; origin: generated-evolved-origin.

```
_haar_packet_energy (lines 360–375): 0360 def _haar_packet_energy(values: np.ndarray, *, levels: int = 3) -> tuple[np.ndarray, float]:
0361 packets = [np.asarray(values, dtype=np.float64).reshape(-1)]
0362 for _ in range(levels):
0363     next_packets: list[np.ndarray] = []
0364     for packet in packets:
0365         if len(packet) < 2:
0366             next_packets.extend([packet, packet * 0.0])
0367         continue
0368         even_len = (len(packet) // 2) * 2
0369         paired = packet[:even_len].reshape(-1, 2)
0370         avg = paired.mean(axis=1)
0371         diff = 0.5 * (paired[:, 0] - paired[:, 1])
0372         next_packets.extend([avg, diff])
0373     packets = next_packets
0374     energy = np.asarray([float(np.mean(np.square(packet))) for packet in packets], dtype=np.float64)
0375     return energy, _entropy(energy)

_topology_proxy (lines 378–396): 0378 def _topology_proxy(series: np.ndarray) -> dict[str, float]:
0379     values = np.asarray(series, dtype=np.float64).reshape(-1)
0380     if len(values) < 4:
0381         return {"nn_scale": 0.0, "loop_area": 0.0, "persistence_spread": 0.0}
0382     emb = np.column_stack([values[:-2], values[1:-1], values[2:]])
0383     if len(emb) > 32:
0384         emb = emb[np.linspace(0, len(emb) - 1, 32).astype(np.int64)]
0385     diff = emb[:, None, :] - emb[None, :, :]
0386     dist = np.sqrt(np.sum(diff * diff, axis=2) + 1e-12)
0387     dist += np.eye(len(emb)) * 1e9
0388     nn = np.min(dist, axis=1)
0389     cov = np.cov(emb.T)
0390     eig = np.sort(np.linalg.eigvalsh(cov + np.eye(cov.shape[0]) * 1e-9))[:-1]
0391     loop_area = math.sqrt(max(float(eig[0] * eig[1]), 0.0)) if len(eig) > 1 else 0.0
0392     return {
0393         "nn_scale": float(np.median(nn)),
0394         "loop_area": loop_area,
0395         "persistence_spread": float(np.percentile(nn, 90) - np.percentile(nn, 10)),
0396     }

_dmd_summary (lines 399–414): 0399 def _dmd_summary(values: np.ndarray) -> dict[str, float]:
0400     matrix = np.asarray(values, dtype=np.complex128)
0401     if matrix.ndim == 1:
0402         x = matrix[:-1]
0403         y = matrix[1:]
0404     else:
0405         x = matrix[:-1].reshape(matrix.shape[0] - 1, -1)
0406         y = matrix[1:].reshape(matrix.shape[0] - 1, -1)
0407     denom = np.vdot(x, x) + 1e-9
0408     alpha = np.vdot(x, y) / denom
0409     residual = np.linalg.norm(y - alpha * x) / (np.linalg.norm(y) + 1e-9)
0410     return {
0411         "growth": float(np.abs(alpha)),
0412         "frequency": float(np.angle(alpha)),
0413         "residual": float(np.real(residual)),
0414     }
```

E. Mixed-Origin EI Selection and Lock

Source: detection/advanced_main_run_detectors.py; origin: mixed-origin.

run_advanced_main_run_detectors (lines 2310–2777): 2310 def run_advanced_main_run_detectors(

```

2311 data_root: Path,
2312 out_root: Path,
2313 *,
2314 folds: int = 5,
2315 seed: int = 202605210136,
2316 force: bool = False,
2317 candidate_limit: int = 128,
2318 evolution_rounds: int = 5,
2319 search_profile: str | SearchProfile = "balanced",
2320 feature_cache: Path | None = None,
2321 evolution_sample_rows: int = 4500,
2322 selection_lock: Path | None = None,
2323 score_locked_only: bool = False,
2324 write_component_scores: bool = False,
2325 ) -> dict[str, Any]:
2326     profile = _resolve_search_profile(search_profile)
2327     if candidate_limit < profile.min_candidate_limit:
2328         raise ValueError(
2329             f"candidate_limit must be at least {profile.min_candidate_limit} "
2330             f"for search profile {profile.name}"
2331         )
2332     if out_root.exists():
2333         if not force:
2334             raise FileExistsError(f"{out_root} already exists; pass --force to replace it")
2335         shutil.rmtree(out_root)
2336     out_root.mkdir(parents=True, exist_ok=True)
2337
2338     data = load_advanced_main_run_data(
2339         data_root,
2340         search_profile=profile,
2341         feature_cache=feature_cache,
2342     )
2343     train_cv = data.split == "train_cv"
2344     holdout = data.split == "holdout"
2345     specs = _make_candidate_specs(data, candidate_limit=candidate_limit)
2346     leaderboard: list[dict[str, Any]] = []
2347     trace_path = out_root / "evolution_trace.jsonl"
2348     trace_csv_path = out_root / "evolution_trace.csv"
2349     evolution_trace_rows: list[dict[str, Any]] = []
2350     running_best_trace_row: dict[str, Any] | None = None
2351     candidate_train_scores: dict[str, np.ndarray] = {}
2352     candidate_train_raws: dict[str, np.ndarray] = {}
2353     spec_by_id = {spec.candidate_id: spec for spec in specs}
2354
2355     def add_evolution_trace(stage: str, row: dict[str, Any]) -> None:
2356         nonlocal running_best_trace_row
2357         if running_best_trace_row is None or _trace_sort_tuple(row) > _trace_sort_tuple(
2358             running_best_trace_row
2359         ):
2360             running_best_trace_row = dict(row)
2361         assert running_best_trace_row is not None
2362         evolution_trace_rows.append(
2363             _make_evolution_trace_row(
2364                 candidate_index=len(evolution_trace_rows) + 1,
2365                 stage=stage,
2366                 row=row,

```

```

2367 running_best=running_best_trace_row,
2368 )
2369 )
2370
2371 for spec in specs:
2372     raw = _candidate_oof_raw(
2373         data,
2374         spec,
2375         folds=folds,
2376         seed=seed,
2377         evolution_rounds=evolution_rounds,
2378         evolution_sample_rows=evolution_sample_rows,
2379     )
2380     candidate_train_rows[spec.candidate_id] = raw
2381     for calibrator in CALIBRATORS:
2382         calibration = _fit_calibration(raw[train_cv], data.y[train_cv], calibrator)
2383         scores = np.zeros(len(data.records), dtype=np.float64)
2384         scores[train_cv] = calibration.transform(raw[train_cv])
2385         candidate_id = f"{spec.candidate_id}.{calibrator}"
2386         objective = _advanced_objective(
2387             data.y[train_cv], scores[train_cv], data.phases[train_cv]
2388         )
2389         row = {
2390             "candidate_id": candidate_id,
2391             "base_candidate_id": spec.candidate_id,
2392             "candidate_type": "advanced_candidate",
2393             "family": spec.family,
2394             "subset_name": spec.subset_name,
2395             "head": spec.head,
2396             "calibrator": calibrator,
2397             "feature_count": len(spec.feature_indices),
2398             "selection_split": "train_cv",
2399             "holdout_rows_used_for_selection": 0,
2400             "objective": f"{objective['objective']:.9f}",
2401             "train_cv_average_precision": f"{objective['average_precision']:.9f}",
2402             "train_cv_roc_auc": f"{objective['roc_auc']:.9f}",
2403             "train_cv_f1": f"{objective['f1']:.9f}",
2404             "train_cv_false_positive_rate": f"{objective['false_positive_rate']:.9f}",
2405             "initial_take_up_average_precision": (
2406                 f"{objective['initial_take_up_average_precision']:.9f}"
2407             ),
2408             "threshold": f"{objective['threshold']:.9f}",
2409             "calibration_info": json.dumps(calibration.info, sort_keys=True),
2410         }
2411         leaderboard.append(row)
2412         candidate_train_scores[candidate_id] = scores
2413         add_evolution_trace("base_candidate_search", row)
2414
2415     surface_scores, surface_rows = _surface_candidate_scores(data)
2416     leaderboard.extend(surface_rows)
2417     candidate_train_scores.update(surface_scores)
2418     for row in surface_rows:
2419         add_evolution_trace("surface_control", row)
2420     leaderboard.sort(
2421         key=lambda row: (
2422             _safe_float(row["objective"]),
2423             _safe_float(row["train_cv_average_precision"]),

```

```

2424 _safe_float(row["train_cv_roc_auc"]),
2425 row["candidate_id"],
2426 ),
2427 reverse=True,
2428 )
2429 meta_specs, meta_scores, meta_rows = _build_meta_fusion_candidates(
2430 data,
2431 candidate_train_scores,
2432 leaderboard,
2433 profile,
2434 seed=seed,
2435 evolution_rounds=evolution_rounds,
2436 evolution_sample_rows=evolution_sample_rows,
2437 )
2438 leaderboard.extend(meta_rows)
2439 candidate_train_scores.update(meta_scores)
2440 for row in meta_rows:
2441 add_evolution_trace("meta_fusion_search", row)
2442 leaderboard.sort(
2443 key=lambda row: (
2444 _safe_float(row["objective"]),
2445 _safe_float(row["train_cv_average_precision"]),
2446 _safe_float(row["train_cv_roc_auc"]),
2447 row["candidate_id"],
2448 ),
2449 reverse=True,
2450 )
2451 if len(leaderboard) < candidate_limit:
2452 raise AssertionError(
2453 f"expected at least {candidate_limit} candidates, got {len(leaderboard)}"
2454 )
2455 leaderboard = leaderboard[: len(leaderboard)]
2456 eligible_selected = [
2457 row
2458 for row in leaderboard
2459 if row.get("candidate_type") == "meta_fusion" and row.get("calibrator") == "geodesic_odds"
2460 ]
2461 if not eligible_selected:
2462 eligible_selected = [row for row in leaderboard if row.get("candidate_type") == "meta_fusion"]
2463 selected = eligible_selected[0] if eligible_selected else leaderboard[0]
2464 selected_rank = leaderboard.index(selected) + 1
2465 if selection_lock is not None:
2466 if not selection_lock.exists():
2467 raise FileNotFoundError(f"missing selection lock override: {selection_lock}")
2468 lock_payload = _read_json(selection_lock)
2469 override_id = str(lock_payload.get("selected_candidate_id", ""))
2470 if override_id:
2471 override = next(
2472 (row for row in leaderboard if row["candidate_id"] == override_id), None
2473 )
2474 if override is None:
2475 raise ValueError(
2476 f"selection lock requested candidate {override_id!r}, which is not on the leaderboard"
2477 )
2478 selected = override
2479 selected_rank = leaderboard.index(selected) + 1
2480 selected_base = str(selected["base_candidate_id"])
2481 selected_calibrator_name = str(selected["calibrator"])

```

```

2482
2483 for row in evolution_trace_rows:
2484     is_selected = row["candidate_id"] == selected["candidate_id"]
2485     row["selected_by_cv"] = is_selected
2486     row["final_selected"] = is_selected
2487     with trace_path.open("w", encoding="utf-8") as trace:
2488         for row in evolution_trace_rows:
2489             trace.write(json.dumps(row, sort_keys=True) + "\n")
2490         _write_csv(trace_csv_path, evolution_trace_rows)
2491         _write_csv(out_root / "candidate_leaderboard.csv", leaderboard)
2492     selection_lock = {
2493         "dataset_profile": DATASET_PROFILE,
2494         "advanced_output_profile": ADVANCED_OUTPUT_PROFILE,
2495         "search_profile": profile.name,
2496         "selected_candidate_id": selected["candidate_id"],
2497         "selected_candidate_type": selected.get("candidate_type", "advanced_candidate"),
2498         "base_candidate_id": selected_base,
2499         "calibrator": selected_calibrator_name,
2500         "selection_split": "train_cv",
2501         "holdout_rows_used_for_selection": 0,
2502         "candidate_count": len(leaderboard),
2503         "candidate_limit": candidate_limit,
2504         "train_cv_rank": selected_rank,
2505         "selection_policy": (
2506             "best train/CV calibrated meta-fusion artifact when available; "
2507             "single-component candidates remain internal controls"
2508         ),
2509         "train_cv_objective": selected["objective"],
2510         "train_cv_average_precision": selected["train_cv_average_precision"],
2511         "train_cv_roc_auc": selected["train_cv_roc_auc"],
2512         "evolution_trace": {
2513             "schema_version": EVOLUTION_TRACE_SCHEMA_VERSION,
2514             "path": "evolution_trace.jsonl",
2515             "csv_path": "evolution_trace.csv",
2516             "trace_row_count": len(evolution_trace_rows),
2517             "trace_basis": "true_evaluation_order",
2518         },
2519         "split_isolation_policy": {
2520             "candidate_selection_split": "train_cv",
2521             "calibration_split": "train_cv",
2522             "threshold_split": "train_cv",
2523             "holdout_rows_used_for_candidate_selection": 0,
2524             "holdout_rows_used_for_calibration": 0,
2525             "holdout_rows_used_for_threshold_selection": 0,
2526             "holdout_evaluation_policy": "after_selection_lock_only",
2527         },
2528     }
2529     if selected.get("candidate_type") == "meta_fusion" and selected_base in meta_specs:
2530         meta = meta_specs[selected_base]
2531         selection_lock["meta_fusion"] = {
2532             "optimizer": meta.optimizer,
2533             "top_k": meta.top_k,
2534             "component_ids": list(meta.component_ids),
2535             "weights": list(meta.weights),
2536         }
2537     _write_json(out_root / "selection_lock.json", selection_lock)
2538

```

```

2539 selected_method, selected_scores, selected_details = _score_selected_candidate(
2540 data,
2541 selected,
2542 spec_by_id,
2543 meta_specs,
2544 candidate_train_raws,
2545 folds=folds,
2546 seed=seed,
2547 evolution_rounds=evolution_rounds,
2548 evolution_sample_rows=evolution_sample_rows,
2549 )
2550
2551 component_rows: list[dict[str, Any]] = []
2552 ablation_rows: list[dict[str, Any]] = []
2553 component_summary: dict[str, Any] = {
2554 "selected_kind": selected_details.get("selected_kind", "advanced_candidate"),
2555 "component_count": int(selected_details.get("component_count", 1)),
2556 }
2557 if (
2558 write_component_scores
2559 or selection_lock is not None
2560 or selected_details.get("selected_kind") == "meta_fusion"
2561 ):
2562 component_rows, ablation_rows, component_summary = _build_selected_candidate_transparency(
2563 data,
2564 selected,
2565 selected_details,
2566 spec_by_id,
2567 meta_specs,
2568 candidate_train_raws,
2569 folds=folds,
2570 seed=seed,
2571 evolution_rounds=evolution_rounds,
2572 evolution_sample_rows=evolution_sample_rows,
2573 )
2574
2575 method_scores = {
2576 **data.surface_scores,
2577 selected_method: selected_scores,
2578 }
2579 performance_rows, performance_summary = _advanced_performance_rows(
2580 data,
2581 method_scores,
2582 candidate_train_scores,
2583 )
2584 performance_summary["advanced_selection"] = {
2585 "selected_method": selected_method,
2586 "selected_candidate_id": selected["candidate_id"],
2587 "selected_candidate_type": selected.get("candidate_type", "advanced_candidate"),
2588 "base_candidate_id": selected_base,
2589 "calibrator": selected_calibrator_name,
2590 "selection_split": "train_cv",
2591 "holdout_rows_used_for_selection": 0,
2592 "candidate_count": len(leaderboard),
2593 "candidate_limit": candidate_limit,
2594 "search_profile": profile.name,
2595 "train_cv_rank": selected_rank,
2596 "selection_policy": selection_lock.get("selection_policy", ""),

```

```

2597 "objective": selected["objective"],
2598 **selected_details,
2599 }
2600 selected_threshold, selected_train_threshold_metrics = _select_threshold(
2601 data.y[train_cv], selected_scores[train_cv]
2602 )
2603 selected_holdout_metrics = _binary_metrics(
2604 data.y[holdout], selected_scores[holdout], selected_threshold
2605 )
2606 selected_holdout_ap = _float_or_none(
2607 _average_precision(data.y[holdout], selected_scores[holdout])
2608 )
2609 selected_holdout_auc = _float_or_none(_roc_auc(data.y[holdout], selected_scores[holdout]))
2610 train_cv_ap = _safe_float(selected["train_cv_average_precision"])
2611 train_cv_auc = _safe_float(selected["train_cv_roc_auc"])
2612 target = BASELINE_TARGETS["layered_fusion_c2"]
2613 expected_full_records = DEFAULT_SCENARIO_GROUPS * len(PHASES)
2614 promotion_metrics_pass = (
2615 train_cv_ap > target["train_cv"]["average_precision"]
2616 and selected_holdout_ap is not None
2617 and selected_holdout_ap > target["holdout"]["average_precision"]
2618 and selected_holdout_auc is not None
2619 and selected_holdout_auc >= target["holdout"]["roc_auc"]
2620 )
2621 promotion_gate = {
2622 "baseline_method": "layered_fusion_c2",
2623 "full_run_record_count_required": expected_full_records,
2624 "record_count": len(data.records),
2625 "train_cv_ap_target": target["train_cv"]["average_precision"],
2626 "holdout_ap_target": target["holdout"]["average_precision"],
2627 "holdout_auc_target": target["holdout"]["roc_auc"],
2628 "selected_train_cv_average_precision": train_cv_ap,
2629 "selected_train_cv_roc_auc": train_cv_auc,
2630 "selected_holdout_average_precision": selected_holdout_ap,
2631 "selected_holdout_roc_auc": selected_holdout_auc,
2632 "status": (
2633 "pass"
2634 if len(data.records) == expected_full_records and promotion_metrics_pass
2635 else "smoke_not_promoted"
2636 if len(data.records) != expected_full_records
2637 else "not_promoted"
2638 ),
2639 }
2640
2641 prediction_rows: list[dict[str, Any]] = []
2642 for idx, record in enumerate(data.records):
2643 prediction_rows.append(
2644 {
2645 "record_id": record["record_id"],
2646 "scenario_group_id": record["scenario_group_id"],
2647 "time_lock_id": record["time_lock_id"],
2648 "phase_id": record["phase_id"],
2649 "split_role": record["split_role"],
2650 "cv_fold": record["cv_fold"],
2651 "label_id": int(record["label_id"]),
2652 "selected_candidate_id": selected["candidate_id"],
2653 "advanced_score": f"{float(selected_scores[idx]):.9f}",
2654 "threshold": f"{float(selected_threshold):.9f}",

```

```

2655 "binary_prediction": int(float(selected_scores[idx]) >= selected_threshold),
2656 "calibration_role": (
2657     "cv_fold_scored"
2658     if record["split_role"] == "train_cv"
2659     else "holdout_scored_once_after_selection"
2660 ),
2661 "high_resolution_xku_cuas": f"{float(data.surface_scores['high_resolution_xku_cuas'][idx]):.9f}",
2662 "tactical_s_band_aesa": f"{float(data.surface_scores['tactical_s_band_aesa'][idx]):.9f}",
2663 "gbad_3d4d_cueing": f"{float(data.surface_scores['gbad_3d4d_cueing'][idx]):.9f}",
2664 "distributed_acoustic_cue": f"{float(data.surface_scores[ACOUSTIC_VIEW_ID][idx]):.9f}",
2665 "layered_fusion_c2": f"{float(data.surface_scores[FUSION_VIEW_ID][idx]):.9f}",
2666 }
2667 )
2668
2669 feature_manifest = {
2670     "dataset_profile": DATASET_PROFILE,
2671     "advanced_output_profile": ADVANCED_OUTPUT_PROFILE,
2672     "method_family": ADVANCED_METHOD_ID,
2673     "strict_open_claim_boundary": (
2674         "synthetic public-proxy experiment; no measured-platform, proprietary-equivalent, "
2675         "classified-fidelity, or deployment-performance claim"
2676     ),
2677     "candidate_generation_policy": {
2678         "search_profile": profile.name,
2679         "requested_candidate_limit": candidate_limit,
2680         "actual_candidate_count": len(leaderboard),
2681         "base_candidate_count": len(specs),
2682         "calibrators": list(CALIBRATORS),
2683         "heads": sorted({spec.head for spec in specs}),
2684         "selection_metric": "AP + 0.25*AUC - FPR_penalty - weak_initial_take_up_penalty",
2685         "second_stage_fusion": {
2686             "enabled": True,
2687             "optimizer": "clean_room_shade_multi_resolution_de",
2688             "top_k_values": list(profile.fusion_top_ks),
2689             "pool_size": profile.fusion_pool_size,
2690             "nonnegative_weights": True,
2691         },
2692     },
2693     "clean_room_inspiration": {
2694         "source_note": (
2695             "Vex math was used only as high-level clean-room inspiration; no source code, "
2696             "imports, symbols, or copied implementation are used."
2697         ),
2698         "implemented_families": [
2699             "spectral_conditioning",
2700             "marchenko_pastur_shrinkage_mass",
2701             "multi_lag_temporal_decorrelation",
2702             "ssa_hankel_rank_features",
2703             "morlet_wavelet_scattering_summaries",
2704             "koopman_edmd_style_residuals",
2705             "diffusion_laplacian_eigengaps",
2706             "persistent_laplacian_takens_topology_proxies",
2707             "train_cv_quantile_transport",
2708             "copula_rank_geometry",
2709             "wasserstein_style_prototype_distances",
2710             "information_geometric_alpha_means",
2711         ],

```

```

2712 },
2713 "split_isolation_policy": {
2714 "candidate_selection_split": "train_cv",
2715 "calibration_split": "train_cv",
2716 "threshold_split": "train_cv",
2717 "passive_rank_reference_split": "train_cv",
2718 "holdout_rows_used_for_feature_selection": 0,
2719 "holdout_rows_used_for_candidate_selection": 0,
2720 "holdout_rows_used_for_calibration": 0,
2721 "holdout_rows_used_for_threshold_selection": 0,
2722 "holdout_evaluation_policy": "single selected winner scored after internal-CV lock",
2723 "selection_lock_path": "selection_lock.json",
2724 },
2725 "model_feature_denylist": list(MODEL_FEATURE_DENYLIST),
2726 "feature_count": len(data.feature_names),
2727 "feature_columns": data.feature_names,
2728 "branch_feature_counts": {
2729 branch: len(indices) for branch, indices in sorted(data.branch_indices.items())
2730 },
2731 **data.manifest,
2732 }
2733 quality = {
2734 "dataset_profile": DATASET_PROFILE,
2735 "advanced_output_profile": ADVANCED_OUTPUT_PROFILE,
2736 "record_count": len(data.records),
2737 "candidate_count": len(leaderboard),
2738 "selected_candidate_id": selected["candidate_id"],
2739 "selected_candidate_type": selected.get("candidate_type", "advanced_candidate"),
2740 "search_profile": profile.name,
2741 "holdout_isolation_status": "pass",
2742 "selection_lock_status": "written_before_holdout_scoring",
2743 "candidate_selection_holdout_record_count": 0,
2744 "calibration_holdout_record_count": 0,
2745 "threshold_holdout_record_count": 0,
2746 "selected_holdout_evaluation_count": 1,
2747 "selected_train_cv_threshold_metrics": selected_train_threshold_metrics,
2748 "selected_holdout_threshold_metrics": selected_holdout_metrics,
2749 "promotion_gate": promotion_gate,
2750 "outputs": {
2751 "candidate_leaderboard": "candidate_leaderboard.csv",
2752 "selection_lock": "selection_lock.json",
2753 "advanced_feature_manifest": "advanced_feature_manifest.json",
2754 "evolution_trace": "evolution_trace.jsonl",
2755 "evolution_trace_csv": "evolution_trace.csv",
2756 "advanced_predictions": "advanced_predictions.csv",
2757 "performance_metrics": "performance_metrics.csv",
2758 "performance_summary": "performance_summary.json",
2759 "fusion_quality_report": "fusion_quality_report.json",
2760 },
2761 "status": "pass",
2762 }
2763
2764 if component_rows:
2765 _write_csv(out_root / "selected_component_scores.csv", component_rows)
2766 _write_csv(out_root / "selected_component_ablations.csv", ablation_rows)
2767 _write_json(out_root / "selected_component_aliases.json", component_summary)
2768 quality["outputs"]["selected_component_scores"] = "selected_component_scores.csv"

```

```

2769 quality["outputs"]["selected_component_ablations"] = "selected_component_ablations.csv"
2770 quality["outputs"]["selected_component_aliases"] = "selected_component_aliases.json"
2771
2772 _write_json(out_root / "advanced_feature_manifest.json", feature_manifest)
2773 _write_csv(out_root / "advanced_predictions.csv", prediction_rows)
2774 _write_csv(out_root / "performance_metrics.csv", performance_rows)
2775 _write_json(out_root / "performance_summary.json", performance_summary)
2776 _write_json(out_root / "fusion_quality_report.json", quality)
2777 return quality

```

F. Human-Origin Paper Evidence Generation

Source: detection/paper_evidence_major_upgrade_v1.py; origin: human-origin.

build_paper_evidence (lines 3734–4515): 3734 def build_paper_evidence(roots: EvidenceRoots, *, strict_ei_trace: bool = False) -> di

```

3735 if roots.out_root.exists():
3736     shutil.rmtree(roots.out_root)
3737     roots.out_root.mkdir(parents=True, exist_ok=True)
3738
3739 training_manifest = _read_json(roots.training_root / "dataset_manifest.json")
3740 training_quality = _read_json(roots.training_root / "quality_report.json")
3741 scenarios = _read_csv_rows(roots.training_root / "scenario_manifest.csv")
3742 records = _read_csv_rows(roots.training_root / "records.csv")
3743 radar_model_card = _build_radar_model_card(roots.training_root, scenarios)
3744 radar_model_detail_rows = _radar_model_detail_rows(radar_model_card)
3745 sensor_archetype_cards = _build_public_sensor_archetype_cards()
3746 regional_hard_negative_taxonomy = _build_regional_hard_negative_taxonomy()
3747 scenario_balance = _scenario_balance_payload(scenarios)
3748 scenario_balance_by_dimension = _balance_dimension_rows(scenarios, records, positive_only=False)
3749 positive_balance_by_dimension = _balance_dimension_rows(scenarios, records, positive_only=True)
3750 split_summary = _split_summary(scenarios, records)
3751 feedback_coverage = _feedback_coverage_summary()
3752 generative_origin_audit = _run_generative_origin_audit(roots.out_root)
3753 detector_view_schema_evidence = _detector_view_schema_evidence(roots.training_root)
3754 data_processing_trace_rows = _data_processing_trace_rows(split_summary)
3755 simulation_best_practice_rows = _simulation_best_practice_rows()
3756 monte_carlo_setup_rows = _monte_carlo_setup_rows(split_summary)
3757 detector_processing_baseline_rows = _detector_processing_baseline_rows()
3758 fusion_baseline_rows = _fusion_baseline_rows()
3759 ei_objective_rows = _ei_objective_rows()
3760 cli_reproduction_commands = _cli_reproduction_commands()
3761 source_appendix_hashes = _source_appendix_hashes()
3762
3763 holdout_records = [row for row in records if row.get("split_role") == "holdout"]
3764 holdout_scenarios = [row for row in scenarios if row.get("split_role") == "holdout"]
3765
3766 leakage = {
3767     "canary_forbidden_feature_test": _detect_forbidden_features(roots.training_root),
3768     "label_shuffle_sanity": None,
3769     "nearest_neighbor_scan": None,
3770     "metadata_only_baseline": None,
3771     "stratum_only_baseline": None,
3772 }
3773 selected_root = roots.advanced_root
3774 selected_rows = _read_csv_rows(selected_root / "advanced_predictions.csv")
3775 selected_lock = _load_selection_lock(selected_root / "selection_lock.json")
3776 selected_method = str(selected_lock.get("selected_candidate_id", ""))
3777 selected_holdout = [row for row in selected_rows if row.get("split_role") == "holdout"]
3778 selected_labels = np.asarray(

```

```

3779 [_safe_int(row.get("label_id")) for row in selected_holdout], dtype=np.int8
3780 )
3781 selected_scores = np.asarray(
3782     [_safe_float(row.get("advanced_score")) for row in selected_holdout], dtype=np.float64
3783 )
3784 rng = np.random.default_rng(BOOTSTRAP_SEED)
3785 shuffled = selected_labels.copy()
3786 rng.shuffle(shuffled)
3787 leakage["label_shuffle_sanity"] = {
3788     "selected": _metric_bundle(
3789         selected_labels,
3790         selected_scores,
3791         _holdout_threshold_from_performance(
3792             selected_root / "performance_metrics.csv", selected_method
3793         ),
3794     ),
3795     "shuffled_labels": _metric_bundle(
3796         shuffled,
3797         selected_scores,
3798         _holdout_threshold_from_performance(
3799             selected_root / "performance_metrics.csv", selected_method
3800         ),
3801     ),
3802 }
3803 selected_feature_rows = [
3804     {
3805         "record_id": row.get("record_id", ""),
3806         "scenario_group_id": row.get("scenario_group_id", ""),
3807         "high_resolution_xku_cuas": row.get("high_resolution_xku_cuas", ""),
3808         "tactical_s_band_aesa": row.get("tactical_s_band_aesa", ""),
3809         "gbad_3d4d_cueing": row.get("gbad_3d4d_cueing", ""),
3810         "distributed_acoustic_cue": row.get("distributed_acoustic_cue", ""),
3811         "layered_fusion_c2": row.get("layered_fusion_c2", ""),
3812     }
3813     for row in selected_rows
3814     if row.get("split_role") in {"train_cv", "holdout"}
3815 ]
3816 train_group_ids = {
3817     scenario["scenario_group_id"]
3818     for scenario in scenarios
3819     if scenario.get("split_role") == "train_cv"
3820 }
3821 holdout_group_ids = {
3822     scenario["scenario_group_id"]
3823     for scenario in scenarios
3824     if scenario.get("split_role") == "holdout"
3825 }
3826 train_feature_rows = [
3827     row for row in selected_feature_rows if row.get("scenario_group_id", "") in train_group_ids
3828 ]
3829 holdout_feature_rows = [
3830     row
3831     for row in selected_feature_rows
3832     if row.get("scenario_group_id", "") in holdout_group_ids
3833 ]
3834 leakage["nearest_neighbor_scan"] = _nearest_neighbor_scan(
3835     train_feature_rows,

```

```

3836 holdout_feature_rows,
3837 [
3838 "high_resolution_xku_cuas",
3839 "tactical_s_band_aesa",
3840 "gbad_3d4d_cueing",
3841 "distributed_acoustic_cue",
3842 "layered_fusion_c2",
3843 ],
3844 )
3845
3846 leakage.update(_stratum_and_metadata_baselines(scenarios, holdout_records))
3847
3848 eval_summary = _build_evaluation_summary(roots.baseline_root, roots.advanced_root)
3849 phase_method_ladder_rows = _phase_method_ladder_rows(eval_summary)
3850 ei_evolution_evidence = build_ei_evolution_evidence(
3851 roots.advanced_root,
3852 roots.out_root,
3853 strict_trace=strict_ei_trace,
3854 )
3855 component_transparency = _load_component_transparency(roots.advanced_root)
3856 anchor_summary = _load_anchor_summary(roots.anchor_root)
3857 normalized_anchor_comparison = _normalized_anchor_comparison(anchor_summary)
3858 anchor_distance_diagnostics = _anchor_distance_diagnostics(anchor_summary)
3859 modality_transparency = _build_modality_transparency(
3860 selected_rows,
3861 component_transparency.get("component_scores", []),
3862 component_transparency.get("component_ablations", []),
3863 float(eval_summary.get("selected_threshold", float("nan"))),
3864 )
3865 comparable_ablation_summary = _comparable_ablation_rows(eval_summary, modality_transparency)
3866 selected_component_human_weights = _selected_component_human_weights(
3867 component_transparency.get("component_scores", [])
3868 )
3869 primary_kpi_rows = _primary_kpi_rows(eval_summary)
3870 main_kpi_gain_rows = _main_kpi_gain_rows(eval_summary)
3871 false_alarm_rows = _false_alarm_breakdown(
3872 holdout_scenarios,
3873 [row for row in selected_rows if row.get("split_role") == "holdout"],
3874 np.asarray(
3875 [
3876 _safe_float(row.get("advanced_score"))
3877 for row in selected_rows
3878 if row.get("split_role") == "holdout"
3879 ],
3880 dtype=np.float64,
3881 ),
3882 float(eval_summary.get("selected_threshold", float("nan"))),
3883 )
3884 false_positive_tables = _false_positive_method_tables(
3885 scenarios=scenarios,
3886 baseline_root=roots.baseline_root,
3887 advanced_root=roots.advanced_root,
3888 selected_method=selected_method,
3889 selected_threshold=float(eval_summary.get("selected_threshold", float("nan"))),
3890 eval_summary=eval_summary,
3891 )
3892 selected_threshold_confusion_rows = _selected_threshold_confusion_rows(
3893 false_positive_tables["method_summary_rows"]

```

```

3894 )
3895 engineered_intelligence_rows = _engineered_intelligence_rows(
3896 eval_summary, component_transparency, modality_transparency
3897 )
3898 source_pack_bird_coverage = _source_pack_bird_coverage_summary()
3899 public_proxy_positive_class_card = _public_proxy_positive_class_card()
3900 public_proxy_model_detail_rows = _public_proxy_model_detail_rows(
3901 public_proxy_positive_class_card
3902 )
3903 environment_impairment_model_rows = _environment_impairment_model_rows(radar_model_card)
3904 regional_bird_library = _build_regional_bird_library(
3905 false_positive_tables["family_rows"], false_positive_tables["method_summary_rows"]
3906 )
3907
3908 selected_component_path = roots.advanced_root / "selected_component_scores.csv"
3909 component_score_rows = _read_csv_rows(selected_component_path)
3910 if component_score_rows:
3911 selected_component_columns = sorted(
3912 {row.get("component_id", "") for row in component_score_rows if row.get("component_id")}
3913 )
3914 else:
3915 selected_component_columns = []
3916
3917 payload = {
3918 "version": PAPER_EVIDENCE_VERSION,
3919 "training_manifest": training_manifest,
3920 "training_quality": training_quality,
3921 "radar_model_card": radar_model_card,
3922 "radar_model_detail_rows": radar_model_detail_rows,
3923 "sensor_archetype_cards": sensor_archetype_cards,
3924 "regional_hard_negative_taxonomy": regional_hard_negative_taxonomy,
3925 "scenario_balance": scenario_balance,
3926 "scenario_balance_by_dimension": scenario_balance_by_dimension,
3927 "positive_balance_by_dimension": positive_balance_by_dimension,
3928 "split_summary": split_summary,
3929 "detector_view_schema_evidence": detector_view_schema_evidence,
3930 "data_processing_trace_rows": data_processing_trace_rows,
3931 "simulation_best_practice_rows": simulation_best_practice_rows,
3932 "monte_carlo_setup_rows": monte_carlo_setup_rows,
3933 "detector_processing_baseline_rows": detector_processing_baseline_rows,
3934 "fusion_baseline_rows": fusion_baseline_rows,
3935 "ei_objective_rows": ei_objective_rows,
3936 "phase_method_ladder_rows": phase_method_ladder_rows,
3937 "cli_reproduction_commands": cli_reproduction_commands,
3938 "source_appendix_hashes": source_appendix_hashes,
3939 "ei_evolution_summary": ei_evolution_evidence.get("summary", {}),
3940 "ei_evolution_evidence": ei_evolution_evidence,
3941 "leakage_diagnostics": leakage,
3942 "evaluation_summary": eval_summary,
3943 "component_transparency": component_transparency,
3944 "modality_transparency": modality_transparency,
3945 "comparable_ablation_summary": comparable_ablation_summary,
3946 "selected_component_human_weights": selected_component_human_weights,
3947 "anchor_summary": anchor_summary,
3948 "normalized_anchor_comparison": normalized_anchor_comparison,
3949 "anchor_distance_diagnostics": anchor_distance_diagnostics,
3950 "primary_kpi_rows": primary_kpi_rows,
3951 "main_kpi_gain_rows": main_kpi_gain_rows,

```

```

3952 "group_level_operating_metrics": eval_summary.get("group_operating_metrics", []),
3953 "selected_threshold_confusion_matrix": selected_threshold_confusion_rows,
3954 "selected_component_columns": selected_component_columns,
3955 "false_alarm_family_breakdown": false_alarm_rows,
3956 "false_alarm_by_method_family": false_positive_tables["family_rows"],
3957 "top_method_false_positive_frequency": false_positive_tables["method_summary_rows"],
3958 "regional_bird_library": regional_bird_library,
3959 "source_pack_bird_coverage_summary": source_pack_bird_coverage,
3960 "engineered_intelligence": engineered_intelligence_rows,
3961 "public_proxy_positive_class_card": public_proxy_positive_class_card,
3962 "public_proxy_model_detail_rows": public_proxy_model_detail_rows,
3963 "environment_impairment_model_rows": environment_impairment_model_rows,
3964 "feedback_coverage": feedback_coverage,
3965 "generative_origin_audit": generative_origin_audit,
3966 "source_roots": {
3967     "training_root": str(roots.training_root),
3968     "baseline_root": str(roots.baseline_root),
3969     "advanced_root": str(roots.advanced_root),
3970     "anchor_root": str(roots.anchor_root),
3971 },
3972 }
3973
3974 _write_json(roots.out_root / "paper_evidence_manifest.json", payload)
3975 _write_json(roots.out_root / "radar_model_card.json", radar_model_card)
3976 _write_json(roots.out_root / "radar_model_detail_rows.json", radar_model_detail_rows)
3977 _write_csv(
3978     roots.out_root / "radar_model_detail_rows.csv",
3979     radar_model_detail_rows,
3980     ["branch", "detail_key", "detail_value", "unit", "basis"],
3981 )
3982 _write_json(roots.out_root / "sensor_archetype_cards.json", sensor_archetype_cards)
3983 _write_json(
3984     roots.out_root / "regional_hard_negative_taxonomy.json", regional_hard_negative_taxonomy
3985 )
3986 _write_json(roots.out_root / "scenario_balance.json", scenario_balance)
3987 _write_json(
3988     roots.out_root / "scenario_balance_by_dimension.json", scenario_balance_by_dimension
3989 )
3990 _write_json(
3991     roots.out_root / "positive_balance_by_dimension.json", positive_balance_by_dimension
3992 )
3993 _write_json(roots.out_root / "split_summary.json", split_summary)
3994 _write_csv(
3995     roots.out_root / "scenario_balance.csv",
3996     scenario_balance["grouped_rows"],
3997 [
3998     "split_role",
3999     "label_role",
4000     "site_archetype_id",
4001     "range_band",
4002     "target_aspect",
4003     "noise_regime",
4004     "hard_negative_role",
4005     "count",
4006 ],
4007 )
4008 balance_fields = [
4009     "dimension",

```

```

4010 "value",
4011 "split_role",
4012 "label_role",
4013 "count_unit",
4014 "count",
4015 "positive_only",
4016 ]
4017 _write_csv(
4018 roots.out_root / "scenario_balance_by_dimension.csv",
4019 scenario_balance_by_dimension,
4020 balance_fields,
4021 )
4022 _write_csv(
4023 roots.out_root / "positive_balance_by_dimension.csv",
4024 positive_balance_by_dimension,
4025 balance_fields,
4026 )
4027 _write_csv(
4028 roots.out_root / "false_alarm_family_breakdown.csv",
4029 false_alarm_rows,
4030 [
4031 "family",
4032 "count",
4033 "false_alarm_count",
4034 "false_alarm_rate",
4035 "near_threshold_count",
4036 "near_threshold_rate",
4037 "mean_score",
4038 "p95_score",
4039 "max_score",
4040 ],
4041 )
4042 _write_csv(
4043 roots.out_root / "sensor_archetype_cards.csv",
4044 sensor_archetype_cards,
4045 [
4046 "family",
4047 "paper_label",
4048 "public_role_envelope",
4049 "public_anchors",
4050 "observable_families",
4051 "detector_visible_features",
4052 "excluded_truth_fields",
4053 "limitations",
4054 "prohibited_inferences",
4055 ],
4056 )
4057 _write_json(
4058 roots.out_root / "detector_view_schema_evidence.json", detector_view_schema_evidence
4059 )
4060 _write_csv(
4061 roots.out_root / "detector_view_schema_evidence.csv",
4062 detector_view_schema_evidence,
4063 [
4064 "view_id",
4065 "path",
4066 "status",
4067 "audit_header_status",

```

```

4068 "audit_only_fields",
4069 "forbidden_model_fields",
4070 "allowed_detector_feature_families",
4071 "allowed_detector_feature_count",
4072 "model_schema_note",
4073 ],
4074 )
4075 _write_json(roots.out_root / "data_processing_trace_rows.json", data_processing_trace_rows)
4076 _write_csv(
4077 roots.out_root / "data_processing_trace_rows.csv",
4078 data_processing_trace_rows,
4079 [
4080 "stage",
4081 "input_artifacts",
4082 "output_artifacts",
4083 "reviewer_check",
4084 "claim_boundary",
4085 ],
4086 )
4087 _write_json(
4088 roots.out_root / "simulation_best_practice_rows.json", simulation_best_practice_rows
4089 )
4090 _write_csv(
4091 roots.out_root / "simulation_best_practice_rows.csv",
4092 simulation_best_practice_rows,
4093 ["practice", "implementation", "evidence_artifact", "claim_boundary"],
4094 )
4095 _write_json(roots.out_root / "monte_carlo_setup_rows.json", monte_carlo_setup_rows)
4096 _write_csv(
4097 roots.out_root / "monte_carlo_setup_rows.csv",
4098 monte_carlo_setup_rows,
4099 ["stage", "modeled_dimension", "reviewer_check", "boundary"],
4100 )
4101 _write_json(
4102 roots.out_root / "detector_processing_baseline_rows.json",
4103 detector_processing_baseline_rows,
4104 )
4105 _write_csv(
4106 roots.out_root / "detector_processing_baseline_rows.csv",
4107 detector_processing_baseline_rows,
4108 ["lane", "processing", "input_view", "comparison_role"],
4109 )
4110 _write_json(roots.out_root / "fusion_baseline_rows.json", fusion_baseline_rows)
4111 _write_csv(
4112 roots.out_root / "fusion_baseline_rows.csv",
4113 fusion_baseline_rows,
4114 ["component", "input", "fusion_role", "known_risk"],
4115 )
4116 _write_json(roots.out_root / "ei_objective_rows.json", ei_objective_rows)
4117 _write_csv(
4118 roots.out_root / "ei_objective_rows.csv",
4119 ei_objective_rows,
4120 ["objective", "what_it_rewards", "current_role"],
4121 )
4122 _write_json(roots.out_root / "phase_method_ladder_rows.json", phase_method_ladder_rows)
4123 _write_csv(
4124 roots.out_root / "phase_method_ladder_rows.csv",
4125 phase_method_ladder_rows,

```

```

4126 [
4127 "method",
4128 "method_label",
4129 "phase",
4130 "fixed_fpr_recall",
4131 "average_precision",
4132 "roc_auc",
4133 "positive_count",
4134 "negative_count",
4135 "claim_boundary",
4136 ],
4137 )
4138 _write_json(roots.out_root / "cli_reproduction_commands.json", cli_reproduction_commands)
4139 _write_csv(
4140 roots.out_root / "cli_reproduction_commands.csv",
4141 cli_reproduction_commands,
4142 ["stage", "command"],
4143 )
4144 _write_json(roots.out_root / "source_appendix_hashes.json", source_appendix_hashes)
4145 _write_csv(
4146 roots.out_root / "source_appendix_hashes.csv",
4147 source_appendix_hashes,
4148 [
4149 "block_id",
4150 "path",
4151 "origin",
4152 "symbols",
4153 "line_start",
4154 "line_end",
4155 "sha256",
4156 "status",
4157 ],
4158 )
4159 _write_csv(
4160 roots.out_root / "regional_hard_negative_taxonomy.csv",
4161 regional_hard_negative_taxonomy,
4162 [
4163 "family",
4164 "regional_subfamily",
4165 "wingbeat_hz_proxy",
4166 "prop_cadence_hz_proxy",
4167 "velocity_mps_proxy",
4168 "flock_size_proxy",
4169 "site_archetype",
4170 "seasonality_flag",
4171 "expected_confusion_mechanism",
4172 "paper_boundary",
4173 ],
4174 )
4175 _write_csv(
4176 roots.out_root / "modality_transparency.csv",
4177 modality_transparency["modality_rows"],
4178 [
4179 "view",
4180 "kind",
4181 "threshold_basis",
4182 "threshold",
4183 "average_precision",

```

```

4184 "roc_auc",
4185 "fixed_fpr_recall",
4186 "precision",
4187 "recall",
4188 "false_positive_rate",
4189 "f1",
4190 "ece",
4191 "tp",
4192 "fp",
4193 "tn",
4194 "fn",
4195 "source_columns",
4196 ],
4197 )
4198 _write_csv(
4199 roots.out_root / "ablation_transparency.csv",
4200 modality_transparency["ablation_rows"],
4201 [
4202 "view",
4203 "kind",
4204 "component_scope",
4205 "average_precision",
4206 "roc_auc",
4207 "f1",
4208 "brier_score",
4209 "ece",
4210 ],
4211 )
4212 _write_csv(
4213 roots.out_root / "comparable_ablation_summary.csv",
4214 comparable_ablation_summary,
4215 [
4216 "ablation",
4217 "reader_label",
4218 "diagnostic_type",
4219 "ap",
4220 "delta_ap",
4221 "recall_at_leq_lpct_fpr",
4222 "delta_recall_at_leq_lpct_fpr",
4223 "f1",
4224 "ece",
4225 "metric_basis",
4226 "comparability_note",
4227 ],
4228 )
4229 _write_csv(
4230 roots.out_root / "selected_component_human_weights.csv",
4231 selected_component_human_weights,
4232 [
4233 "audit_id",
4234 "family",
4235 "modality",
4236 "view",
4237 "calibrator",
4238 "weight",
4239 "cumulative_weight",
4240 "component_group",
4241 "raw_component_id",

```

```

4242 ],
4243 )
4244 _write_csv(
4245 roots.out_root / "normalized_anchor_comparison.csv",
4246 normalized_anchor_comparison,
4247 ["feature", "unitless_basis", "q10_z", "q50_z", "q90_z", "count", "claim_boundary"],
4248 )
4249 _write_csv(
4250 roots.out_root / "anchor_distance_diagnostics.csv",
4251 anchor_distance_diagnostics,
4252 [
4253 "feature",
4254 "diagnostic",
4255 "measured_anchor_value",
4256 "synthetic_reference_value",
4257 "distance",
4258 "distance_basis",
4259 "status",
4260 "allowed_claim_level",
4261 ],
4262 )
4263 _write_csv(
4264 roots.out_root / "primary_kpi_table.csv",
4265 primary_kpi_rows,
4266 ["method", "point_estimate", "lcb95", "target_fpr", "basis"],
4267 )
4268 _write_csv(
4269 roots.out_root / "main_kpi_gain_table.csv",
4270 main_kpi_gain_rows,
4271 [
4272 "metric",
4273 "prior_fusion_baseline",
4274 "ei_candidate",
4275 "absolute_change",
4276 "relative_change",
4277 "relative_percent",
4278 "direction",
4279 ],
4280 )
4281 _write_csv(
4282 roots.out_root / "group_level_operating_metrics.csv",
4283 eval_summary.get("group_operating_metrics", []),
4284 [
4285 "method",
4286 "threshold_source",
4287 "threshold",
4288 "positive_groups",
4289 "negative_groups",
4290 "group_tp",
4291 "group_fp",
4292 "group_tn",
4293 "group_fn",
4294 "group_recall",
4295 "group_false_alarm_rate",
4296 "bootstrap_note",
4297 ],
4298 )
4299 _write_csv(

```

```

4300 roots.out_root / "selected_threshold_confusion_matrix.csv",
4301 selected_threshold_confusion_rows,
4302 [
4303 "method",
4304 "method_label",
4305 "threshold_source",
4306 "threshold",
4307 "record_count",
4308 "positive_count",
4309 "negative_count",
4310 "tp",
4311 "fp",
4312 "tn",
4313 "fn",
4314 "precision",
4315 "recall",
4316 "f1",
4317 "false_positive_rate",
4318 "average_precision",
4319 "roc_auc",
4320 ],
4321 )
4322 _write_csv(
4323 roots.out_root / "false_alarm_by_method_family.csv",
4324 false_positive_tables["family_rows"],
4325 [
4326 "method",
4327 "method_label",
4328 "threshold_source",
4329 "threshold",
4330 "average_precision",
4331 "roc_auc",
4332 "record_count",
4333 "negative_count",
4334 "family",
4335 "family_count",
4336 "false_alarm_count",
4337 "false_alarm_rate",
4338 "near_threshold_count",
4339 "near_threshold_rate",
4340 "fp_per_1000_negatives",
4341 "fp_burden_vs_locked_candidate",
4342 ],
4343 )
4344 _write_csv(
4345 roots.out_root / "top_method_false_positive_frequency.csv",
4346 false_positive_tables["method_summary_rows"],
4347 [
4348 "rank",
4349 "method",
4350 "method_label",
4351 "threshold_source",
4352 "threshold",
4353 "average_precision",
4354 "recall_at_leq_1pct_fpr",
4355 "selected_threshold_fp_count",
4356 "fp",
4357 "selected_threshold_fp_rate",

```

```

4358 "fp_per_1000_negatives",
4359 "top_fp_family",
4360 "top_fp_family_fp_count",
4361 "fp_burden_vs_locked_candidate",
4362 "negative_count",
4363 "positive_count",
4364 "record_count",
4365 "tp",
4366 "tn",
4367 "fn",
4368 "precision",
4369 "recall",
4370 "f1",
4371 "roc_auc",
4372 "false_positive_rate",
4373 ],
4374 )
4375 _write_csv(
4376 roots.out_root / "regional_bird_library.csv",
4377 regional_bird_library,
4378 [
4379 "family",
4380 "regional_subfamily",
4381 "wingbeat_hz_proxy",
4382 "prop_cadence_hz_proxy",
4383 "velocity_mps_proxy",
4384 "flock_size_proxy",
4385 "site_archetype",
4386 "seasonality_flag",
4387 "expected_confusion_mechanism",
4388 "paper_boundary",
4389 "site_prevalence",
4390 "body_size_rcs_stress",
4391 "flock_behavior",
4392 "false_alarm_count_total",
4393 "near_threshold_count_total",
4394 "top_method_false_alarm_family",
4395 "top_method_false_alarm_count",
4396 ],
4397 )
4398 _write_csv(
4399 roots.out_root / "source_pack_bird_coverage_summary.csv",
4400 source_pack_bird_coverage,
4401 ["family_slug", "display_name", "path", "status", "object_pack"],
4402 )
4403 _write_csv(
4404 roots.out_root / "engineered_intelligence_transparency.csv",
4405 engineered_intelligence_rows,
4406 ["stage", "summary", "evidence_artifact", "status"],
4407 )
4408 _write_json(roots.out_root / "feedback_coverage.json", feedback_coverage)
4409 _write_csv(
4410 roots.out_root / "feedback_coverage_matrix.csv",
4411 feedback_coverage.get("rows", []),
4412 [
4413 "tip",
4414 "item_id",
4415 "actionable_item",

```

```

4416 "status",
4417 "evidence_location",
4418 "notes",
4419 ],
4420 )
4421 _write_text(
4422 roots.out_root / "feedback_coverage_matrix.md",
4423 _feedback_coverage_markdown(feedback_coverage.get("rows", [])),
4424 )
4425 _write_json(roots.out_root / "leakage_diagnostics.json", leakage)
4426 _write_json(roots.out_root / "evaluation_summary.json", eval_summary)
4427 _write_json(roots.out_root / "anchor_summary.json", anchor_summary)
4428 _write_json(roots.out_root / "modality_transparency.json", modality_transparency)
4429 _write_json(roots.out_root / "comparable_ablation_summary.json", comparable_ablation_summary)
4430 _write_json(
4431 roots.out_root / "selected_component_human_weights.json", selected_component_human_weights
4432 )
4433 _write_json(roots.out_root / "normalized_anchor_comparison.json", normalized_anchor_comparison)
4434 _write_json(roots.out_root / "anchor_distance_diagnostics.json", anchor_distance_diagnostics)
4435 _write_json(
4436 roots.out_root / "group_level_operating_metrics.json",
4437 eval_summary.get("group_operating_metrics", []),
4438 )
4439 _write_json(roots.out_root / "regional_bird_library.json", regional_bird_library)
4440 _write_json(
4441 roots.out_root / "source_pack_bird_coverage_summary.json", source_pack_bird_coverage
4442 )
4443 _write_json(
4444 roots.out_root / "engineered_intelligence_transparency.json", engineered_intelligence_rows
4445 )
4446 _write_json(
4447 roots.out_root / "selected_threshold_confusion_matrix.json",
4448 selected_threshold_confusion_rows,
4449 )
4450 _write_json(
4451 roots.out_root / "false_alarm_by_method_family.json",
4452 false_positive_tables["family_rows"],
4453 )
4454 _write_json(
4455 roots.out_root / "top_method_false_positive_frequency.json",
4456 false_positive_tables["method_summary_rows"],
4457 )
4458 _write_json(
4459 roots.out_root / "public_proxy_positive_class_card.json",
4460 public_proxy_positive_class_card,
4461 )
4462 _write_json(roots.out_root / "main_kpi_gain_table.json", main_kpi_gain_rows)
4463 _write_json(
4464 roots.out_root / "public_proxy_model_detail_rows.json",
4465 public_proxy_model_detail_rows,
4466 )
4467 _write_csv(
4468 roots.out_root / "public_proxy_model_detail_rows.csv",
4469 public_proxy_model_detail_rows,
4470 [
4471 "model_item",
4472 "public_proxy_assumption",
4473 "radar_consequence",

```

```

4474 "evidence_basis",
4475 "prohibited_inference",
4476 ],
4477 )
4478 _write_json(
4479 roots.out_root / "environment_impairment_model_rows.json",
4480 environment_impairment_model_rows,
4481 )
4482 _write_csv(
4483 roots.out_root / "environment_impairment_model_rows.csv",
4484 environment_impairment_model_rows,
4485 [
4486 "family",
4487 "generated_proxy",
4488 "radar_review_role",
4489 "source_values",
4490 "claim_boundary",
4491 ],
4492 )
4493
4494 if component_transparency.get("selection_lock"):
4495 _write_json(
4496 roots.out_root / "selected_candidate_lock.json",
4497 component_transparency["selection_lock"],
4498 )
4499 if component_transparency.get("aliases"):
4500 _write_json(
4501 roots.out_root / "selected_component_aliases.json", component_transparency["aliases"]
4502 )
4503 if component_transparency.get("component_ablations"):
4504 _write_csv(
4505 roots.out_root / "selected_component_ablations.csv",
4506 component_transparency["component_ablations"],
4507 list(component_transparency["component_ablations"][0]),
4508 )
4509 if component_transparency.get("component_scores"):
4510 _write_csv(
4511 roots.out_root / "selected_component_scores.csv",
4512 component_transparency["component_scores"],
4513 list(component_transparency["component_scores"][0]),
4514 )
4515 return payload

```

G. IP Escrow and Redaction Boundary

The escrow excerpt is a duplicate review artifact. It is not required to reproduce the paper KPI; claim-critical code remains visible in repository source and in the listings above.

```

escrow digest 24f7ed187ca72b64f04a41fd822c2eb78ff33dff6a972320d75c1abeeb4649ad
metadata target/paper/source_appendix/ip_escrow_manifest.json

```

REFERENCES

- [1] M. I. Skolnik, *Radar Handbook*, 3rd ed. New York, NY, USA: McGraw-Hill, 2008.
- [2] M. A. Richards, *Fundamentals of Radar Signal Processing*, 2nd ed. New York, NY, USA: McGraw-Hill Education, 2014.
- [3] H. Rohling, "Radar CFAR thresholding in clutter and multiple target situations," *IEEE Transactions on Aerospace and Electronic Systems*, vol. AES-19, no. 4, pp. 608–621, Jul. 1983.
- [4] A. Karlsson, "Radar measurements on drones, birds and humans with a 77GHz FMCW sensor," Zenodo, 2021. [Online]. Available: <https://zenodo.org/records/5896641>
- [5] D. Gusland, J. M. Christiansen, B. Torvik, F. Fioranelli, S. Z. Gurbuz, and M. Ritchie, "Open radar initiative: Large scale dataset for benchmarking of micro-doppler recognition algorithms," in *2021 IEEE Radar Conference (RadarConf21)*. IEEE, 2021.

- [6] I. Roldan, C. R. del Blanco, A. Duque Quevedo, F. Ibanez Urzaiz, J. Gismero Menoyo, A. Asensio Lopez, D. Berjon, F. Jaureguizar, and N. Garcia, "Dopplernet: A convolutional neural network for recognising targets in real scenarios using a persistent range-doppler radar," *IET Radar, Sonar & Navigation*, vol. 14, no. 4, pp. 593–600, 2020.
- [7] S.-K. Han and Y.-H. Jung, "A time-synchronized multi-sensor drone dataset acquired from multiple radars and RF receiver," *Scientific Data*, vol. 13, no. 407, 2026.
- [8] S. Rahman and D. A. Robertson, "Radar micro-doppler signatures of drones and birds at K-band and W-band," *Scientific Reports*, vol. 8, no. 1, p. 17396, 2018.
- [9] P. Molchanov, R. I. A. Harmanny, J. J. M. de Wit, K. Egiazarian, and J. Astola, "Classification of small UAVs and birds by micro-doppler signatures," *International Journal of Microwave and Wireless Technologies*, vol. 6, no. 3–4, pp. 435–444, 2014.
- [10] M. Ritchie, F. Fioranelli, H. Borrión, and H. Griffiths, "Multistatic micro-doppler radar feature extraction for classification of unloaded/loaded micro-drones," *IET Radar, Sonar & Navigation*, vol. 11, no. 1, pp. 116–124, 2017.
- [11] V. C. Chen, *The Micro-Doppler Effect in Radar*. Norwood, MA, USA: Artech House, 2011.
- [12] D. Tahmouh, "Review of micro-doppler signatures," *IET Radar, Sonar & Navigation*, vol. 9, no. 9, pp. 1140–1146, 2015.
- [13] M. Ezuma, O. Ozdemir, C. K. Anjinappa, W. A. Gulzar, and I. Guvenc, "Micro-UAV detection with a low-grazing angle millimeter wave radar," in *2019 IEEE Radio and Wireless Symposium (RWS)*. IEEE, 2019, pp. 1–4.
- [14] K. D. Ward, R. J. A. Tough, and S. Watts, *Sea Clutter: Scattering, the K Distribution and Radar Performance*, 2nd ed. London, UK: Institution of Engineering and Technology, 2013.
- [15] S. M. Kay, *Fundamentals of Statistical Signal Processing, Volume 2: Detection Theory*. Upper Saddle River, NJ, USA: Prentice Hall, 1998.
- [16] H. L. Van Trees, *Detection, Estimation, and Modulation Theory, Part III: Radar-Sonar Signal Processing and Gaussian Signals in Noise*. New York, NY, USA: Wiley, 2001.
- [17] P. Swerling, "Probability of detection for fluctuating targets," *IRE Transactions on Information Theory*, vol. 6, no. 2, pp. 269–308, Apr. 1960.
- [18] D. C. Schleher, *MTI and Pulsed Doppler Radar*. Norwood, MA, USA: Artech House, 1991.
- [19] Blighter Surveillance Systems, "A400 Series Radars," Product information page, 2026. [Online]. Available: <https://blighter.com/products/a400-series-radars/>
- [20] RTX Raytheon, "KuRFS: Ku-band radio frequency system," Product information page, 2026. [Online]. Available: <https://www.rtx.com/raytheon/news/lang/ro/capabilities/products/counter-uas/sensors/kurfis>
- [21] DRS RADA Technologies, "Next-gen multi-mission hemispheric radar (nMHR)," Product information page, 2026. [Online]. Available: <https://www.drsrcada.com/products/nmhr>
- [22] Saab, "Giraffe 1X," Product information page, 2026. [Online]. Available: <https://www.saab.com/products/giraffe-1x>
- [23] RTX Raytheon, "Sentinel radar," Product information page, 2026. [Online]. Available: <https://www.rtx.com/raytheon/what-we-do/land/sentinel-radar>
- [24] D. L. Hall and J. Llinas, "An introduction to multisensor data fusion," *Proceedings of the IEEE*, vol. 85, no. 1, pp. 6–23, 1997.
- [25] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. New York, NY, USA: Wiley, 2001.
- [26] S. Blackman and R. Popoli, *Design and Analysis of Modern Tracking Systems*. Norwood, MA, USA: Artech House, 1999.
- [27] R. P. S. Mahler, *Statistical Multisource-Multitarget Information Fusion*. Norwood, MA, USA: Artech House, 2007.
- [28] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, pp. 468–475, 2024.
- [29] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, "The AI scientist: Towards fully automated open-ended scientific discovery," arXiv:2408.06292, 2024. [Online]. Available: <https://arxiv.org/abs/2408.06292>
- [30] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha, "The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search," arXiv:2504.08066, 2025. [Online]. Available: <https://arxiv.org/abs/2504.08066>
- [31] A. Novikov, P. Veličković, B. Balle *et al.*, "AlphaEvolve: A coding agent for scientific and algorithmic discovery," arXiv:2506.13131, 2025. [Online]. Available: <https://arxiv.org/abs/2506.13131>
- [32] G. S. Fishman, *Monte Carlo: Concepts, Algorithms, and Applications*. New York, NY, USA: Springer, 1996.
- [33] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *The Journal of Chemical Physics*, vol. 21, no. 6, pp. 1087–1092, 1953.
- [34] R. Y. Rubinstein and D. P. Kroese, *Simulation and the Monte Carlo Method*, 3rd ed. Hoboken, NJ, USA: Wiley, 2016.
- [35] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, 1995, pp. 1137–1143.
- [36] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [37] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proceedings of the 23rd International Conference on Machine Learning*. ACM, 2006, pp. 233–240.
- [38] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015.
- [39] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*. Cambridge, MA, USA: MIT Press, 1999, pp. 61–74.
- [40] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proceedings of the 22nd International Conference on Machine Learning*. ACM, 2005, pp. 625–632.
- [41] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 1321–1330.
- [42] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [43] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [44] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [45] W. L. Oberkampff and C. J. Roy, *Verification and Validation in Scientific Computing*. Cambridge, UK: Cambridge University Press, 2010.
- [46] P. J. Roache, *Verification and Validation in Computational Science and Engineering*. Albuquerque, NM, USA: Hermosa Publishers, 1998.
- [47] R. D. Peng, "Reproducible research in computational science," *Science*, vol. 334, no. 6060, pp. 1226–1227, 2011.
- [48] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alche Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.